



ReactJS

Programmation Avancée

Formation avancée — Hooks, Redux, Performances, Animations & i18n

Hooks Avancés

Redux Toolkit

Optimisation

Animations

i18n

TypeScript

Développeurs web & Architectes — Niveau Bachelor → Master

À propos de la formation

Informations pratiques — Formation ReactJS Programmation Avancée

01

DURÉE

3 jours — 21 heures

Sessions intensives de 7h par jour.
9h–12h30 et 14h–17h30.

02

MODALITÉS

Présentiel ou Distanciel

Paris La Défense · Lyon · Nantes
Classe à distance synchrone disponible.

03

PUBLIC VISÉ

Développeurs React

Développeurs web Front-End et Full-Stack.
Architectes logiciels souhaitant approfondir React.

04

RÉFÉRENCE & TARIF

Réf. RJS — 2 120 € H.T.

Note participants : 4,6 / 5
Sessions : 27 mai · 8 juil. · 16 sep. · 30 nov. 2026

Objectifs pédagogiques

Compétences visées à l'issue de la formation

01

Maîtriser les concepts avancés

HOC, Portals, Context API, Custom Hooks et patterns avancés pour des applications robustes.

03

Améliorer la qualité du code

TypeScript, tests Jest+RTL, Storybook, couverture de code $\geq 70\%$.

05

Maîtriser les animations

React Transition Group, Framer Motion, transitions de routes fluides.

02

Optimiser les performances

React.lazy, Suspense, useMemo, useCallback, SSR avec Next.js pour des apps ultra-rapides.

04

Intégrer l'écosystème Redux

Redux Toolkit, Redux Form, Redux Persist, middleware personnalisé.

06

Internationaliser une application

react-i18next, pluralisation, formats de dates/nombres, changement de langue dynamique.

Public & Prérequis

À qui s'adresse cette formation ?

Public cible

Développeurs web Front-End et Full-Stack

Souhaitant maîtriser les concepts avancés de ReactJS pour des applications complexes.

Architectes logiciels

En charge de la conception et de l'optimisation d'architectures front-end modernes.

Chefs de projet technique

Voulant comprendre les enjeux de performance et l'écosystème React actuel.

Prérequis indispensables

✓ Expérience React + Redux

Avoir déjà travaillé sur un projet React réel et comprendre le flux de données Redux.

✓ Formation TJS (recommandée)

Avoir suivi la formation "ReactJS, maîtriser le framework" (Réf. TJS) ou niveau équivalent.

✓ JavaScript ES6+

Maîtrise des arrow functions, destructuring, modules, et de l'asynchronisme (async/await).

Module 1

Introduction & Écosystème

Rappels ES6+ essentiels

Les syntaxes modernes à maîtriser avant d'aborder React avancé

SYNTAXES FONDAMENTALES

Arrow functions

```
const add = (a, b) => a + b;
```

Destructuring

```
const { name, age } = user;
```

Spread / Rest operator

```
const merged = { ...obj1, ...obj2 };
```

MODULES & ASYNCHRONISME

Modules (ESM)

```
import React from 'react';
```

Async / Await

```
const data = await fetchData();
```

Optional chaining

```
const city = user?.address?.city;
```

Rappels ES6+ — Destructuring & Spread

ReactJS Avancé — Module approfondi

CONCEPTS CLÉS

Le destructuring en ES6 permet d'extraire de manière déclarative et concise des valeurs dans des structures itérables ou des objets, favorisant l'immuabilité en évitant les mutations profondes et en facilitant la gestion des états locaux ou props dans React. Le spread/rest exploite le protocole d'itérabilité pour décomposer ou regrouper des collections, optimisant la réconciliation en créant de nouveaux objets ou tableaux plutôt que de modifier l'existant, ce qui prévient les effets secondaires et améliore les performances via la comparaison référentielle. Leur usage combiné doit être manié avec soin pour éviter la création excessive d'objets intermédiaires, impactant la mémoire et le garbage collector, tout en assurant une clarté sémantique et une maintenabilité accrues dans la composition des composants et la propagation des props.

IMPLÉMENTATION

Exemple de code

```
// Destructuring
const user = { name: 'Alice', role: 'Admin' };
const { name, role } = user;

// Spread Operator
const arr1 = [1, 2];
const arr2 = [...arr1, 3, 4]; // [1, 2, 3, 4]

// Rest Parameter
const sum = (...args) => args.reduce((a, b) => a + b, 0);
```

Rappels ES6+ — Async/Await et Promises

ReactJS Avancé — Module approfondi

CONCEPTS CLÉS

Les Promises encapsulent la gestion asynchrone en offrant une interface chainable qui facilite la composition et la propagation d'erreurs via un modèle de réconciliation des états (pending, fulfilled, rejected), évitant ainsi le callback hell et améliorant la lisibilité.

Async/await repose sur une syntaxe impérative qui transforme le code asynchrone en séquentiel grâce à la suspension des coroutines via des générateurs internes, tout en permettant une gestion d'erreur synchronisée via try/catch pour une meilleure maintenabilité. Toutefois, l'usage intensif d'async/await peut induire des blocages séquentiels non désirés, nécessitant une compréhension fine des implications sur la concurrence et la parallélisation des tâches asynchrones.

IMPLÉMENTATION

Exemple de code

```
// Utilisation de Promises
fetch('/api/data')
  .then(res => res.json())
  .then(data => console.log(data))
  .catch(err => console.error(err));
```


Rappels ES6+ — Async/Await et Promises

ReactJS Avancé — Module approfondi

CONCEPTS CLÉS

L'usage d'async/await implique une gestion fine des erreurs et l'anticipation des conditions de concurrence implicites. L'écriture séquentielle masque souvent la latence réelle, ce qui peut provoquer des blocages lorsque plusieurs appels asynchrones indépendants sont chaînés inutilement ; il est alors préférable d'introduire des exécutions parallèles via Promise.all pour optimiser les performances. Le pattern async/await favorise une structure plus modulaire et testable des fonctions asynchrones, tout en nécessitant une vigilance accrue sur la propagation des exceptions et la gestion des ressources, aspects parfois négligés dans les chaînes de Promises classiques.

IMPLÉMENTATION

Exemple de code

```
// Syntaxe Async/Await (Recommandée)
const getData = async () => {
  try {
    const res = await fetch('/api/data');
    const data = await res.json();
    console.log(data);
  } catch (err) {
    console.error(err);
  }
};
```

Principes clés de React

VirtualDOM, JSX et One-way Data Flow

VirtualDOM

React implémente un Virtual DOM, une abstraction immuable du DOM réel, permettant d'effectuer une réconciliation par algorithme de diffing qui identifie de manière optimale les mutations minimales à appliquer, réduisant ainsi les opérations coûteuses sur le DOM natif. Ce processus exploite des heuristiques, notamment la clé des éléments lors du rendu de listes, pour maintenir la stabilité des composants et éviter des re-rendus inutiles, maximisant ainsi la performance tout en conservant un modèle déclaratif. L'architecture favorise l'immuabilité des états et des props, facilitant la comparaison par référence et la mémoïsation, ce qui est crucial pour optimiser la granularité des mises à jour et prévenir les effets secondaires non intentionnels.

JSX

JSX est une abstraction syntaxique qui compile en appels explicites à `React.createElement`, permettant de construire un arbre virtuel d'éléments React exploitable par le moteur de réconciliation. Cette représentation immuable favorise une comparaison efficace des arbres virtuels via l'algorithme de diffing, minimisant les opérations DOM coûteuses et optimisant les mises à jour. En intégrant la logique JavaScript directement dans la déclaration de l'UI, JSX facilite la composition déclarative et la séparation des préoccupations tout en exigeant une rigoureuse gestion des effets de bord pour préserver l'intégrité du rendu.

One-way Data Flow

React impose un flux de données unidirectionnel où les props immuables descendent de la hiérarchie parent-enfant, garantissant une source unique de vérité et facilitant la réconciliation en limitant les effets de bord. La communication ascendante s'effectue via des callbacks fonctionnels passés en props, exploitant des closures pour préserver le contexte lexical tout en évitant les mutations d'état directes, ce qui prévient la corruption des données et facilite la traçabilité des mises à jour. Ce modèle favorise la prédictibilité des états de l'interface, optimise les performances grâce à la mémoïsation ciblée et simplifie le débogage en réduisant les points de synchronisation asynchrones et les effets secondaires non contrôlés.

Le cycle de vie des composants React

ReactJS Avancé — Module approfondi

CONCEPTS CLÉS

Le cycle de vie des composants React repose sur une séquence d'étapes synchrones et asynchrones permettant de gérer la création, la mise à jour et la destruction des instances, où le rendu initial déclenche la phase de réconciliation qui optimise la mise à jour du DOM virtuel en minimisant les mutations réelles. Les méthodes classiques comme `componentDidMount`, `shouldComponentUpdate` et `componentDidUpdate` permettent un contrôle fin du comportement post-rendu et de la prévention des re-rendus inutiles via la comparaison d'état, tandis que l'adoption des Hooks (`useEffect`, `useMemo`, `useCallback`) favorise un découplage fonctionnel, préservant les closures et facilitant la gestion des effets secondaires de manière déclarative et séquentielle. Ces mécanismes favorisent la prévisibilité des états et l'immutabilité, réduisant ainsi les risques de fuites mémoire et améliorant la maintenabilité du code dans des architectures complexes.

IMPLÉMENTATION

Exemple de code

```
import { useEffect, useState } from 'react';

const LifecycleDemo = () => {
  const [count, setCount] = useState(0);

  // Équivalent componentDidMount
  useEffect(() => {
    console.log("Composant monté !");
  }, []);

  // Équivalent componentWillUnmount
  return () => console.log("Composant démonté !");
};
```

Le cycle de vie des composants React

ReactJS Avancé — Module approfondi

CONCEPTS CLÉS

La dépendance des effets dans `useEffect` nécessite une gestion précise du tableau de dépendances pour éviter boucles infinies ou effets obsolètes, ce qui impacte performance et cohérence des états. L'utilisation de fonctions de mise à jour basées sur l'état précédent, comme `setCount(c => c + 1)`, constitue un pattern sécurisé contre les conditions de course lors des mises à jour asynchrones, crucial dans des architectures complexes avec multiples sources d'événements concurrents. La maîtrise fine du cycle de vie via les Hooks permet aussi d'optimiser par la mémorisation conditionnelle et la séparation des effets, réduisant les re-rendus et favorisant une architecture réactive et scalable.

IMPLÉMENTATION

Exemple de code

```
// Équivalent componentDidMount
useEffect(() => {
  console.log(`Count a changé : ${count}`);
  }, [count]); // S'exécute quand count change
return <button onClick={() => setCount(c => c + 1)}>+1</button>;
};
```

Composants fonctionnels vs Composants de classe

ReactJS Avancé — Module approfondi

CONCEPTS CLÉS

Les composants fonctionnels exploitent les hooks pour gérer l'état et le cycle de vie via des closures et des effets déclaratifs, favorisant l'immutabilité et la pureté fonctionnelle, ce qui facilite la réconciliation et l'optimisation par React. À l'inverse, les composants de classe reposent sur un modèle impératif et mutable avec des méthodes de cycle de vie explicites, ce qui peut engendrer des complexités liées à la gestion du `this` et à la surcharge cognitive. La standardisation des composants fonctionnels découle de leur meilleure intégration avec les optimisations de rendu (memoïsation via `React.memo`, `useCallback`) et une meilleure séparation des préoccupations, améliorant la maintenabilité et la testabilité dans des architectures complexes.

IMPLÉMENTATION

Exemple de code

```
// ❌ Composant de Classe (Legacy)
class Counter extends React.Component {
  state = { count: 0 };
  increment = () => {
    this.setState({ count: this.state.count + 1 });
  }
  render() {
    return <button
      onClick={this.increment}>{this.state.count}</button>;
  }
}
```

Composants fonctionnels vs Composants de classe

ReactJS Avancé — Module approfondi

CONCEPTS CLÉS

L'importance des closures dans la gestion d'état via `useState`, notamment l'usage de la fonction de mise à jour basée sur la valeur précédente, qui évite les bugs liés à la concurrence ou aux mises à jour asynchrones. Elle illustre aussi un pattern fonctionnel qui rend le code plus prévisible et testable, en réduisant les effets de bord et en facilitant la composition via des hooks personnalisés. Enfin, le découplage implicite de la logique d'état dans les fonctions favorise l'adoption de paradigmes réactifs plus déclaratifs, accélérant l'intégration de solutions comme Redux Toolkit ou Recoil dans des architectures modulaires.

IMPLÉMENTATION

Exemple de code

```
// ✅ Composant Fonctionnel (Moderne)
const Counter = () => {
  const [count, setCount] = useState(0);
  return <button onClick={() => setCount(c => c +
1)}>{count}</button>;
};
```

L'écosystème ReactJS en 2024

Les outils incontournables pour des applications modernes

Catégorie	Outil recommandé	Usage principal
Bundler & Dev Server	Vite	Build ultra-rapide et serveur de développement avec HMR instantané.
Gestion d'état global	Redux Toolkit	Standard moderne pour Redux, moins de boilerplate, Immer intégré.
Routage	React Router v6	Navigation SPA, gestion des layouts et chargement de données.
Formulaires	React Hook Form	Gestion performante des formulaires avec validation (ex: Zod).
Tests	Jest + React Testing Library	Tests unitaires et d'intégration basés sur le comportement utilisateur.
Typage statique	TypeScript	Sécurité du code, autocomplétion et documentation implicite.
Framework Full-Stack	Next.js	Server-Side Rendering (SSR), Static Site Generation (SSG), SEO.
Animations	Framer Motion	Animations déclaratives complexes et transitions de pages fluides.
Internationalisation	react-i18next	Gestion multilingue, pluralisation et formatage de dates/nombres.

Architecture d'une application React scalable

ReactJS Avancé — Module approfondi

CONCEPTS CLÉS

L'architecture modulaire d'une application React scalable repose sur la colocation des fichiers par fonctionnalité (feature-slicing), optimisant la maintenabilité en limitant le couplage et en facilitant la réconciliation ciblée du Virtual DOM. Organiser les dossiers en `features/`, `components/`, `hooks/`, `store/`, `services/`, `types/`, `utils/` favorise l'isolation des responsabilités et la gestion explicite des dépendances, tout en améliorant la granularité des builds et la memoïsation des composants. Ce découpage explicite permet également d'implémenter plus aisément des stratégies avancées comme le code-splitting et la lazy-loading, tout en assurant une évolutivité contrôlée par la séparation claire des états locaux, globaux et des effets secondaires.

IMPLÉMENTATION

Exemple de code

```
// Architecture Feature-Based recommandée
src/
├── components/      # Composants génériques (UI)
│   ├── Button.tsx
│   └── Modal.tsx
├── features/        # Modules métiers isolés
│   ├── auth/
│   │   ├── components/
│   │   ├── hooks/
│   │   └── api.ts
│   └── dashboard/
```


Architecture d'une application React scalable

ReactJS Avancé — Module approfondi

CONCEPTS CLÉS

L'importance cruciale d'un découpage rigoureux entre hooks, services, store et utils pour éviter l'entrelacement des responsabilités et limiter les effets de bord transversaux qui nuisent à la scalabilité. Placer les custom hooks globaux dans un dossier dédié permet non seulement leur réutilisation cohérente mais aussi d'appliquer des optimisations avancées telles que le memoization fine-grainée des hooks pour réduire les rerenders inutiles. Par ailleurs, séparer clairement les services d'API et la configuration du store facilite l'implémentation de middlewares, la gestion asynchrone des états via Redux-Saga ou Zustand, ainsi que l'intégration progressive de patterns comme la normalisation des données ou le caching centralisé, essentiels pour la performance et la maintenabilité à long terme.

IMPLÉMENTATION

Exemple de code

```
| hooks/           # Custom hooks globaux
| services/        # Appels API génériques
| store/           # Configuration Redux/Zustand
| utils/           # Fonctions utilitaires
```

Server-Side Rendering avec Next.js

ReactJS Avancé — Module approfondi

CONCEPTS CLÉS

Next.js App Router exploite une architecture hybride combinant rendu côté serveur (SSR) et génération statique incrémentale (ISR) via des fonctions spécifiques comme `getServerSideProps` et `getStaticProps`, permettant de choisir précisément entre pré-rendu dynamique et statique selon les besoins métier. SSR génère la page à chaque requête, assurant un contenu toujours à jour, mais induit une latence plus élevée et une charge serveur accrue, alors que ISR optimise la scalabilité en régénérant les pages statiques en arrière-plan selon une fréquence définie, conciliant fraîcheur des données et performances. La décision entre ces stratégies doit s'appuyer sur une analyse fine des contraintes SEO, du taux de changement des données et de la charge serveur, en tirant parti de la réconciliation côté client pour maintenir la cohérence de l'interface sans sacrifier la rapidité d'affichage initiale.

IMPLÉMENTATION

Exemple de code

```
// SSR avec Next.js (App Router)
// page.tsx (S'exécute sur le serveur)
export default async function Page() {
  // Fetch direct dans le composant (Server Component)
  const res = await fetch('https://api.example.com/data', {
    next: { revalidate: 3600 } // Incremental Static Regeneration (ISR)
  });
  const data = await res.json();
```

Server-Side Rendering avec Next.js

ReactJS Avancé — Module approfondi

CONCEPTS CLÉS

Le pattern Server-Side Rendering avec Next.js pose des enjeux cruciaux liés à la granularité du rendu : chaque appel fetch dans un Server Component bloque la génération du HTML, ce qui peut engendrer des goulots d'étranglement si les appels externes sont nombreux ou lents. Une optimisation avancée consiste à découpler la logique de data fetching en hooks ou services réutilisables, tout en tirant parti des mécanismes natifs de Next.js comme la mise en cache et la revalidation automatique pour réduire les requêtes redondantes. Architecturalement, le choix d'intégrer directement des appels API dans des Server Components demande une vigilance accrue sur la gestion des erreurs et des délais d'attente, ainsi qu'une réflexion sur la responsabilité des données dans la chaîne de rendu, soulignant la frontière floue entre backend et frontend dans les architectures modernes.

IMPLÉMENTATION

Exemple de code

```
return (
  <main>
    <h1>Données pré-rendues pour le SEO</h1>
    <ul>
      {data.items.map(item => <li
key={item.id}>{item.name}</li>)}
    </ul>
  </main>
);
```

React Server Components

ReactJS Avancé — Module approfondi

CONCEPTS CLÉS

Les React Server Components exploitent une exécution côté serveur pour générer du markup HTML statique sans embarquer de JavaScript client, ce qui réduit significativement la charge de rendu initial et améliore la performance perçue via la minimisation du bundle client. En orchestrant la séparation stricte entre composants serveurs, qui peuvent accéder directement aux ressources backend sans surcoût réseau ni sérialisation complexe, et composants clients, activés via la directive 'use client', on optimise la réconciliation en limitant la fréquence et la granularité des hydrations nécessaires. Cette architecture impose cependant une gestion rigoureuse de l'état et des effets secondaires, nécessitant l'immutabilité stricte des props et une conception fonctionnelle pour garantir la pureté des Server Components et éviter les anomalies liées à la divergence de contexte d'exécution.

IMPLÉMENTATION

Exemple de code

```
// React Server Components (RSC)
// ✅ Composant Serveur (par défaut dans Next.js App Router)
// Ne charge pas de JS côté client, accès direct à la DB
import db from './database';

export default async function ServerProfile() {
  const user = await db.user.findFirst();
  return <div>{user.name}</div>;
}
```

React Server Components

ReactJS Avancé — Module approfondi

CONCEPTS CLÉS

L'importance cruciale de la distinction explicite entre composants serveurs et clients pour maintenir la cohérence du rendu et éviter les erreurs d'hydratation. Le marquage « use client » introduit un contexte de gestion d'état avec hooks React, ce qui entraîne la nécessité de préserver la compatibilité entre la logique côté client et les parties statiques générées côté serveur. Cette dualité impose des patterns architecturaux hybrides, où la composition fine favorise la délégation des responsabilités : les Server Components gèrent la récupération et la préparation des données lourdes, tandis que les Client Components orchestrent l'interactivité locale. Une bonne pratique avancée consiste à minimiser l'envoi de props dynamiques non sérialisables aux Server Components pour éviter les erreurs de désynchronisation et améliorer le pipeline de streaming, tout en exploitant l'optimisation de rendu incrémental pour réduire les temps de latence perçus.

IMPLÉMENTATION

Exemple de code

```
// ❌ Composant Client (doit être explicite)
'use client';
import { useState } from 'react';

export default function ClientCounter() {
  const [count, setCount] = useState(0);
  return <button onClick={() => setCount(c+1)}>{count}</button>;
}
```

Micro-Frontends avec React

ReactJS Avancé — Module approfondi

CONCEPTS CLÉS

L'architecture Micro-Frontends découple les applications en unités autonomes déployables indépendamment, orchestrées via Module Federation qui exploite le chargement dynamique de modules partagés au runtime pour optimiser la réutilisation sans duplication. Avec Vite et vite-plugin-federation, l'intégration repose sur l'export et l'import asynchrones de composants React encapsulant état et logique métier, tout en garantissant l'isolation des dépendances via des stratégies de résolution partagée strictes pour éviter les conflits de versions. Ce découplage granulaire améliore la scalabilité organisationnelle mais complexifie la gestion transversale de l'état global et la cohérence stylistique, nécessitant des patterns avancés comme la propagation d'événements custom ou l'usage de CSS-in-JS scoped et la synchronisation via Context API ou stores externes.

IMPLÉMENTATION

Exemple de code

```
// Micro-Frontends avec Vite (vite-plugin-federation)
// vite.config.js (Application Hôte)
import federation from "@originjs/vite-plugin-federation";

export default {
  plugins: [
    federation({
```

Micro-Frontends avec React

ReactJS Avancé — Module approfondi

CONCEPTS CLÉS

La configuration fine des remotes et le partage des dépendances exigent une vigilance particulière, car un mauvais paramétrage peut entraîner des duplications silencieuses de React, provoquant des erreurs d'état et des hooks invalides. L'usage explicite de remotes par URL impose une gestion rigoureuse des versions déployées et des stratégies de cache pour éviter les ruptures à l'exécution. L'import asynchrone des composants distants nécessite d'implémenter des fallbacks et des mécanismes de chargement robustes afin d'assurer une expérience utilisateur fluide malgré la latence réseau, tout en préservant l'isolation des contextes pour éviter les fuites d'état entre micro-frontends. Cette orchestration fine invite également à repenser la CI/CD pour intégrer des pipelines indépendants mais synchronisés, garantissant la compatibilité et la stabilité globale du mashup applicatif.

IMPLÉMENTATION

Exemple de code

```
name: 'host-app',
  remotes: {
    remoteApp: 'http://localhost:5001/assets/remoteEntry.js',
  },
  shared: ['react', 'react-dom']
})
]
};

// Utilisation dans l'hôte
import RemoteButton from 'remoteApp/Button';
<RemoteButton />
```

TP 1 — Environnement de développement

Travaux pratiques · Module 1

 **Objectif : Créer une application React+Vite+TypeScript servant de fil rouge pour toute la formation**

Étapes à suivre

1

Créer le projet

```
npm create vite@latest my-react-app -- --template react-ts
```

2

Installer les dépendances

```
cd my-react-app && npm install
```

3

Configurer ESLint + Prettier

```
npm install -D eslint prettier eslint-config-prettier
```

4

Initialiser Git

```
git init && git add . && git commit -m "init"
```

5

Lancer le serveur

```
npm run dev
```

Résultat attendu

Application React fonctionnelle accessible sur <http://localhost:5173> avec TypeScript activé, ESLint configuré et dépôt Git initialisé.

Livrable

Dépôt Git local contenant la structure de base du projet fil rouge, prêt pour les prochains modules.

Pièges fréquents

Ne pas oublier le flag `--template react-ts` pour activer TypeScript dès le départ.

Vérifier que Node.js version ≥ 18 est bien installé sur votre machine.

Module 2

Bonnes pratiques

Typage avec TypeScript

Interfaces de props et composants typés

INTERFACE DE PROPS

```
interface UserCardProps {  
  name: string;  
  age: number;  
  role?: 'admin' | 'user';  
}  
  
const UserCard: React.FC<UserCardProps> = ({  
  name, age, role = "user"  
}) => {  
  return (  
    <div className="card">  
      <h2>{name}</h2>  
      <p>{age} ans · {role}</p>  
    </div>  
  );  
};
```

AVANTAGES TYPESCRIPT

- ✓ **Autocomplétion IDE précise**
L'éditeur connaît exactement les props attendues et leur type.
- ✓ **Détection d'erreurs à la compilation**
Les erreurs de type (ex: passer une string au lieu d'un number) sont détectées avant l'exécution.
- ✓ **Documentation implicite**
L'interface sert de documentation toujours à jour pour les autres développeurs.
- ✓ **Refactoring sécurisé**
Renommer une prop mettra en évidence tous les endroits où elle est utilisée.

TypeScript — Types avancés

ReactJS Avancé — Module approfondi

CONCEPTS CLÉS

Les types génériques permettent d'exprimer des abstractions polymorphes en conservant la traçabilité et la sécurité du typage à la compilation, ce qui favorise la réutilisabilité tout en évitant les castings explicites coûteux en robustesse. L'utilisation combinée des types union et intersection offre un contrôle précis sur les signatures et les variations structurelles, mais nécessite des gardes de type explicites pour assurer la discrimination et éviter les erreurs d'exécution liées à la cohabitation de types incompatibles. Les types utilitaires comme Partial ou Pick exploitent la manipulation de mapped types pour transformer et extraire des sous-ensembles d'interfaces, optimisant ainsi la gestion fine des props dans React tout en renforçant l'immuabilité et la cohérence des contrats d'API.

IMPLÉMENTATION

Exemple de code

```
// Types utilitaires TypeScript
interface User {
  id: number;
  name: string;
  email: string;
}

// Omit : retirer des propriétés
type UserInput = Omit<User, 'id'>;

// Partial : toutes les propriétés optionnelles
type UserUpdate = Partial<User>;

// Record : créer un dictionnaire typé
type UserRoles = Record<string, 'admin' | 'user' | 'guest'>;
```

TypeScript — Typer les événements React

ReactJS Avancé — Module approfondi

CONCEPTS CLÉS

Les événements React sont enveloppés dans des interfaces typées spécifiques (React.ChangeEvent, React.FormEvent, React.MouseEvent) qui exploitent la propagation du type générique pour garantir une stricte correspondance entre l'élément source et le payload événementiel, permettant ainsi une vérification statique robuste et une autocomplétion précise dans les IDE. Le typage des refs via useRef nécessite une gestion fine de la mutabilité implicite et du cycle de vie des éléments DOM ou composants, en combinant généralement les types nullable avec des assertions d'existence pour maîtriser les états transitoires sans compromettre la sécurité mémoire. L'adoption rigoureuse de ces typages favorise l'immuabilité des événements et optimise la réconciliation en limitant les effets secondaires non typés, tout en évitant les pièges liés au widening des types génériques qui pourraient engendrer des erreurs difficiles à traquer à l'exécution.

IMPLÉMENTATION

Exemple de code

```
import React, { useState, ChangeEvent, MouseEvent } from 'react';

const Form = () => {
  const [val, setVal] = useState('');

  // Typer un événement de champ texte
  const handleChange = (e: ChangeEvent<HTMLInputElement>) => {
    setVal(e.target.value);
  };
};
```

TypeScript — Typer les événements React

ReactJS Avancé — Module approfondi

CONCEPTS CLÉS

La gestion des événements asynchrones dans React nécessite un appel explicite à `e.preventDefault()` pour contrôler le comportement natif tout en maintenant une cohérence typée. L'utilisation des types spécifiques comme `MouseEvent<HTMLButtonElement>` permet d'optimiser le DOM virtuel en réduisant les rendus inutiles grâce à des handlers typés précisément plutôt que génériquement. Ce pattern structure clairement la séparation entre la logique événementielle et la gestion d'état, ce qui facilite la testabilité unitaire et l'intégration avec des hooks personnalisés, où un typage strict garantit l'invariance des contrats API internes, renforçant robustesse et maintenabilité des composants complexes.

IMPLÉMENTATION

Exemple de code

```
// Typer un événement de clic
const handleSubmit = (e: MouseEvent<HTMLButtonElement>) => {
  e.preventDefault();
  console.log(val);
};

return (
  <form>
    <input type="text" value={val} onChange={handleChange} />
    <button onClick={handleSubmit}>Envoyer</button>
  </form>
);
```

Tests avec Jest & React Testing Library

Tester le comportement observable plutôt que l'implémentation interne

EXEMPLE DE TEST COMPLET

```
import { render, screen } from '@testing-library/react';
import { UserCard } from './UserCard';

describe('UserCard', () => {
  it('affiche le nom et le rôle', () => {
    render(<UserCard name="Alice" role="admin" />);

    expect(screen.getByText('Alice')).toBeInTheDocument();
    expect(screen.getByText(/admin/i)).toBeInTheDocument();
  });

  it('utilise le rôle par défaut', () => {
    render(<UserCard name="Bob" />);

    expect(screen.getByText(/user/i)).toBeInTheDocument();
  });
});
```

RÈGLES D'OR DES TESTS

Tester le comportement

Se concentrer sur ce que l'utilisateur voit et fait, pas sur l'implémentation interne (state, méthodes privées).

Nommer clairement

```
it('affiche une erreur si le champ est vide')
```

Isoler les tests

Chaque test doit être indépendant. Utiliser `beforeEach` et `afterEach` pour réinitialiser l'état.

Mocker les dépendances

Simuler les appels API (ex: MSW), les modules externes et les timers pour des tests rapides et déterministes.

Viser une bonne couverture

```
npm test -- --coverage
```

Tests — Stratégies et pyramide de tests

ReactJS Avancé — Module approfondi

CONCEPTS CLÉS

La pyramide de tests structure l'effort de vérification en maximisant la couverture des tests unitaires, qui valident des fonctions pures et des composants isolés via la réconciliation virtuelle, minimisant les coûts de maintenance et permettant une détection rapide des régressions. Les tests d'intégration orchestrent plusieurs composants et services, garantissant la cohérence des interactions asynchrones et des effets de bord dans le DOM réel, tout en équilibrant la complexité croissante et le temps d'exécution. Les tests end-to-end reproduisent les scénarios utilisateur finaux via une simulation complète du navigateur, validant les flux critiques mais nécessitant une gestion rigoureuse des états partagés et une orchestration robuste pour limiter leur fragilité et leur coût en temps.

IMPLÉMENTATION

Exemple de code

```
// Exemple de test unitaire avec Jest
import { sum } from './math';

describe('Math utilities', () => {
  it('should add two numbers correctly', () => {
    expect(sum(1, 2)).toBe(3);
    expect(sum(-1, 1)).toBe(0);
  });
});

// Pyramide des tests :
// 1. Tests Unitaires (Jest) : ~70%
// 2. Tests d'Intégration (React Testing Library) : ~20%
// 3. Tests E2E (Playwright/Cypress) : ~10%
```

Tests — Mocking et tests asynchrones

ReactJS Avancé — Module approfondi

CONCEPTS CLÉS

La fonction `jest.mock()` intercepte la résolution des modules importés en remplaçant leurs exports par des mocks contrôlables, ce qui permet d'isoler les unités testées tout en évitant les effets de bord liés aux dépendances externes. L'utilisation de Mock Service Worker (msw) simule au niveau réseau les requêtes HTTP en interceptant les appels `fetch` ou `XMLHttpRequest`, facilitant ainsi des tests end-to-end cohérents et répliquables sans dépendre d'un backend réel, ce qui améliore la fiabilité des tests asynchrones tout en préservant la couche réseau. L'attente des mises à jour asynchrones via `waitFor` repose sur la boucle d'événements JavaScript et la réconciliation de React, en réévaluant périodiquement le DOM virtuel jusqu'à convergence, ce qui évite les tests fragiles basés sur des délais fixes et garantit la synchronisation avec le cycle de rendu effectif.

IMPLÉMENTATION

Exemple de code

```
import { render, screen, waitFor } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import { api } from './api';
import UserProfile from './UserProfile';

// Mock de l'API
jest.mock('./api');

test('loads and displays user data', async () => {
  api.getUser.mockResolvedValueOnce({ name: 'Alice' });

  render(<UserProfile id="1" />);

  // Test asynchrone avec waitFor ou findBy
  const nameElement = await screen.findByText(/Alice/i);
  expect(nameElement).toBeInTheDocument();
});
```


Tests E2E avec Playwright

Automatisation fiable des scénarios utilisateurs avec Playwright

Écrire un test Playwright

```
// import { test, expect } from '@playwright/test';

test('connexion utilisateur', async ({ page }) => {
  await page.goto('/login');
  await page.getByLabel('Email').fill('user@test.com');
  await page.getByLabel('Mot de passe').fill('password');
  await page.getByRole('button', { name: 'Se connecter' }).click();
  await expect(page).toHaveURL('/dashboard');
  await expect(page.getByText('Bienvenue')).toBeVisible();
});
```

Bonnes pratiques E2E

- Utiliser `getByRole/getByLabel` (résistants aux changements CSS)
- Tester les flux critiques uniquement (connexion, achat, inscription)
- Éviter les sélecteurs CSS fragiles (data-testid en dernier recours)
- Utiliser les fixtures pour les données de test
- Intégrer dans CI/CD avec `playwright test --reporter=html`

Playwright vs Cypress : Playwright supporte plusieurs navigateurs, est plus rapide et mieux adapté aux applications modernes.

CI/CD pour les projets React

ReactJS Avancé — Module approfondi

CONCEPTS CLÉS

Un pipeline CI/CD efficace pour un projet React intègre des étapes automatisées de linting, tests unitaires et d'intégration pour assurer la qualité du code avant la compilation, exploitant la réconciliation incrémentale de React pour optimiser les tests ciblés. Le déploiement continu sur des plateformes comme Vercel ou Netlify s'appuie sur des hooks web et des artefacts immuables pour garantir la reproductibilité et la rapidité du rollback, limitant ainsi les dérives environnementales. L'adoption de stratégies de branching avancées telles que GitFlow ou trunk-based development favorise un équilibre entre la stabilité de la branche principale et la rapidité d'intégration, en minimisant les conflits de fusion grâce à une gestion rigoureuse des branches feature et release.

IMPLÉMENTATION

Exemple de code

```
# Exemple de pipeline CI/CD GitHub Actions pour React
name: React CI/CD

on:
  push:
    branches: [ "main" ]

jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - uses: actions/setup-node@v3
        with: { node-version: 18 }
```

CI/CD pour les projets React

ReactJS Avancé — Module approfondi

CONCEPTS CLÉS

L'importance cruciale de l'ordonnancement précis des étapes dans un pipeline CI/CD pour React, où l'utilisation de `npm ci` garantit une installation reproductible et rapide des dépendances, réduisant les risques d'incohérences liées au `lockfile`. L'exécution séquentielle du linter avant les tests évite le gaspillage des ressources sur un code structurellement invalide, tandis que l'intégration des tests unitaires avant la phase de build assure qu'aucune version non validée ne soit déployée, renforçant ainsi la confiance dans le pipeline. Par ailleurs, on soulignera les optimisations possibles via le caching intelligent des dépendances et des artefacts de build pour accélérer les runs successifs, ainsi que l'évolution vers des workflows parallèles ou conditionnels afin d'adapter dynamiquement le pipeline en fonction des changements détectés, tout en limitant la dette technique et en assurant une scalabilité robuste du processus d'intégration continue.

IMPLÉMENTATION

Exemple de code

```
- name: Install dependencies
  run: npm ci

- name: Run Linter
  run: npm run lint

- name: Run Tests
  run: npm run test

- name: Build
  run: npm run build
```

Sécurité dans les applications React

ReactJS Avancé — Module approfondi

CONCEPTS CLÉS

La gestion sécurisée des contenus HTML dans React nécessite une maîtrise fine de `dangerouslySetInnerHTML`, qui contourne la réconciliation virtuelle pour injecter directement du DOM, exposant ainsi à des vulnérabilités XSS si le contenu n'est pas rigoureusement assaini via des parseurs sécurisés et des politiques CSP strictes. La sécurisation des routes en SPA s'appuie sur des Protected Routes implémentées via des gardiens contextuels, exploitant l'état d'authentification immuable pour éviter les re-renders inutiles et réduire la surface d'attaque liée au détournement de redirections ou d'accès non autorisés. La gestion des tokens JWT privilégie l'usage de cookies `httpOnly` afin d'empêcher l'accès via JavaScript, limitant ainsi les risques de fuite par XSS, tout en nécessitant une stratégie robuste de renouvellement et de révocation côté serveur pour éviter les attaques CSRF et garantir un modèle d'authentification sécurisé et scalable.

IMPLÉMENTATION

Exemple de code

```
// Sécurité : Prévention XSS
// React échappe automatiquement les variables :
const userInput = "<script>alert(__STR_0__)</script>";
<div>{userInput}</div> // Affichera le texte brut, pas de script
exécuté

// Danger : dangerouslySetInnerHTML
// Ne JAMAIS utiliser sans sanitize (ex: DOMPurify)
import DOMPurify from 'dompurify';

const rawHTML = "<p>Texte <b>gras</b></p>";
const cleanHTML = DOMPurify.sanitize(rawHTML);

<div dangerouslySetInnerHTML={{ __html: cleanHTML }} />
```

Accessibilité (a11y) dans React

ReactJS Avancé — Module approfondi

CONCEPTS CLÉS

L'intégration des attributs ARIA dans le JSX permet de définir des rôles et propriétés sémantiques statiques ou dynamiques, assurant une communication explicite avec les technologies d'assistance via le DOM virtuel de React, ce qui nécessite une gestion fine pour éviter les conflits entre props contrôlées et attributs ARIA. La gestion programmatique du focus clavier repose sur des références React combinées à des hooks d'effets afin de garantir une navigation séquentielle conforme aux normes WCAG, tout en minimisant les réconciliations inutiles qui pourraient perturber l'état du DOM. Par ailleurs, l'usage de bibliothèques UI headless comme Radix ou Headless UI favorise une séparation claire entre la logique d'accessibilité et la présentation, permettant une composition modulaire et optimisée, tandis que les tests automatisés avec jest-axe appliquent une analyse statique et dynamique des arbres DOM pour détecter les violations d'accessibilité en phase CI/CD, renforçant ainsi la robustesse des interfaces accessibles.

IMPLÉMENTATION

Exemple de code

```
// Accessibilité (a11y) dans React
// Utilisation des attributs ARIA et de la sémantique
const AccessibleButton = ({ onClick, isLoading }) => {
  return (
    <button
      onClick={onClick}
      disabled={isLoading}
      aria-busy={isLoading}
      aria-label="Sauvegarder les modifications"
    >
```

Accessibilité (a11y) dans React

ReactJS Avancé — Module approfondi

CONCEPTS CLÉS

La gestion des contenus dynamiques et la synchronisation avec les lecteurs d'écran sont cruciales pour garantir une accessibilité robuste en conditions réelles. L'utilisation de l'attribut `aria-hidden` sur l'icône de chargement empêche une lecture redondante ou confusionnelle, mais nécessite un contrôle rigoureux pour éviter d'occulter des informations essentielles par inadvertance. La bascule conditionnelle dans le rendu du contenu impose une attention particulière à la stabilité du DOM afin d'éviter des interruptions dans le focus ou des annonces vocales intempestives, ce qui peut être optimisé grâce à des techniques telles que le maintien du même nœud DOM avec des mises à jour internes plutôt qu'un remount complet. Architecturer ces composants en favorisant des hooks personnalisés pour centraliser la logique d'accessibilité et en adoptant des patterns de rendu contrôlé versus non contrôlé améliore la maintenabilité et la testabilité, tout en préservant une expérience utilisateur fluide et conforme aux attentes des technologies d'assistance.

IMPLÉMENTATION

Exemple de code

```
{isLoading ? (  
  <span aria-hidden="true">⌛</span> // Caché aux lecteurs  
  d'écran  
) : (  
  "Sauvegarder"  
  )  
</button>  
);  
};
```

TP 2 — Qualité du code : TypeScript et tests

Travaux pratiques · Module 2



Objectif : Migrer vers TypeScript et mettre en place des tests automatisés sur l'application fil rouge

Étapes à suivre

1

Migrer les composants

Ajouter des interfaces TypeScript pour toutes les props des composants existants.

2

Configurer Jest + RTL

```
npm install -D jest @testing-library/react @testing-library/jest-dom
```

3

Écrire 3 tests unitaires

Tester le composant UserCard, le Header et le formulaire principal.

4

Mesurer la couverture

Lancer les tests avec l'option de couverture et viser au moins 70%.

```
npm test -- --coverage
```

Résultat attendu

Rapport de couverture $\geq 70\%$ généré avec succès, et tous les composants correctement typés avec TypeScript sans erreur de compilation.

Livable

Capture d'écran du rapport de couverture dans le terminal + code source typé pushé sur le dépôt Git.



Pièges fréquents :

Oublier d'installer

@types/react

et @types/node

. Confondre

interface

et type (préférer

interface

pour les props).

Module 3

Techniques & Patterns Avancés



Higher Order Components (HOC)

Pattern : enrichir un composant sans le modifier

DÉFINITION & IMPLÉMENTATION

```
// HOC d'authentification
function withAuth<T extends object>(  
  WrappedComponent: React.ComponentType<T>  
) {  
  return function AuthComponent(props: T) {  
    const { isLoggedIn } = useAuth();  
  
    if (!isLoggedIn) {  
      return <Redirect to="/login" />;  
    }  
  
    return <WrappedComponent {...props} />;  
  };  
}  
  
// Utilisation  
const ProtectedDashboard = withAuth(Dashboard);
```

CAS D'USAGE TYPIQUES

Authentification

Protéger l'accès à certaines routes privées en redirigeant les utilisateurs non connectés.

Logging & Analytics

Enregistrer automatiquement les props reçues ou le cycle de vie (montage/démontage) d'un composant.

Gestion des permissions

Afficher ou masquer des éléments de l'interface en fonction des rôles et droits de l'utilisateur.

Injection de données

Fournir des données spécifiques depuis un store global ou une API directement dans les props du composant enveloppé.

HOC — Composition et cas avancés

ReactJS Avancé — Module approfondi

CONCEPTS CLÉS

La composition de plusieurs High Order Components (HOC) via des fonctions comme `compose()` permet d'enchaîner des transformations de composants en exploitant la réconciliation optimisée de React, mais introduit souvent un empilement profond de wrappers qui complexifie le debugging et la gestion du cycle de vie. Les HOC `withLoading` et `withError` illustrent l'encapsulation des préoccupations transverses, cependant leur usage intensif peut impacter la lisibilité et les performances à cause des nombreux rendus imbriqués ; cela souligne l'importance de préserver l'immutabilité des props et d'optimiser la mémoïsation des composants. La migration vers des Custom Hooks améliore la composition fonctionnelle en tirant parti des closures JavaScript pour partager la logique d'état sans modifier la hiérarchie des composants, évitant ainsi le « wrapper hell » tout en conservant la granularité fine du contrôle sur les effets et le cycle de vie.

IMPLÉMENTATION

Exemple de code

```
// Exemple de Higher Order Component (HOC)
const withAuth = (WrappedComponent) => {
  return (props) => {
    const isAuthenticated = useAuth(); // Hook custom
    if (!isAuthenticated) {
      return <Redirect to="/login" />;
    }

    return <WrappedComponent {...props} />;
  };
};

// Utilisation
const ProtectedDashboard = withAuth(Dashboard);
```

Context API — Conception avancée

ReactJS Avancé — Module approfondi

CONCEPTS CLÉS

L'utilisation de Context API typé avec TypeScript impose une définition stricte des interfaces pour garantir l'intégrité des données partagées et éviter les erreurs de typage au runtime, renforçant ainsi la robustesse du typage statique dans la chaîne de compilation. Séparer explicitement le Provider du Consumer permet d'isoler la logique de distribution d'état du composant consommateur, facilitant la réutilisabilité et le testing, tout en limitant la portée des re-renders via une granularité contrôlée. L'intégration de useMemo dans le Provider prévient la recreation inutile des valeurs contextuelles lors de chaque cycle de rendu, exploitant la mémoïsation pour minimiser les recalculs et optimiser la phase de réconciliation en réduisant les diffusions de mises à jour aux sous-arbres React.

IMPLÉMENTATION

Exemple de code

```
// Context API : Création et Provider
import { createContext, useContext, useState } from 'react';

const ThemeContext = createContext<'light' | 'dark'>('light');

export const ThemeProvider = ({ children }) => {
  const [theme, setTheme] = useState('light');
  return (
    <ThemeContext.Provider value={{ theme, setTheme }}>
      {children}
    </ThemeContext.Provider>
  );
};

// Custom Hook pour consommer le context
export const useTheme = () => useContext(ThemeContext);
```

Portals React — Cas d'usage détaillés

ReactJS Avancé — Module approfondi

CONCEPTS CLÉS

ReactDOM.createPortal permet d'injecter un arbre React dans un nœud DOM distinct de la hiérarchie DOM principale tout en conservant la continuité du contexte React, ce qui évite les problèmes liés au z-index ou à la propagation des styles dans des composants tels que les modales ou tooltips. Sous le capot, les événements émis dans un portal respectent la phase de bubbling React en remontant vers l'arbre React racine, indépendamment de la structure réelle du DOM, ce qui nécessite une gestion fine pour éviter les conflits de gestionnaires d'événements ou les fuites de focus. Architecturer avec des portals impose de maîtriser la synchronisation entre le DOM virtuel et physique, en assurant une unidirectionnalité des données et une gestion rigoureuse des effets de bord pour maintenir l'immuabilité du DOM et garantir des performances optimales via la réconciliation.

IMPLÉMENTATION

Exemple de code

```
import { createPortal } from 'react-dom';

const Modal = ({ children, isOpen, onClose }) => {
  if (!isOpen) return null;

  // Rend le composant à la racine du DOM (hors de l'arbre React
  // courant)
  return createPortal(
    <div className="modal-overlay" onClick={onClose}>
      <div className="modal-content" onClick={e =>
        e.stopPropagation()}>
        {children}
      </div>
    </div>,
    document.getElementById('modal-root') // Nœud cible dans
    index.html
  );
};
```

Les React Hooks fondamentaux

Programmation fonctionnelle et gestion du cycle de vie avec les Hooks

Hook	Rôle principal	Exemple d'usage typique
useState	État local du composant	Compteur, champs de formulaire, toggle (ouvert/fermé).
useEffect	Effets de bord (cycle de vie)	Requêtes API (fetch), abonnements, timers, manipulation du DOM.
useContext	Consommer un contexte global	Thème (clair/sombre), données d'authentification, langue (i18n).
useRef	Référence DOM / Valeur persistante	Focus sur un input, stocker un ID de timer, conserver une valeur sans re-render.
useMemo	Mémoiser une valeur calculée	Calculs coûteux, filtrage ou tri de grandes listes de données.
useCallback	Mémoiser une fonction	Passer des callbacks stables aux composants enfants pour éviter leurs re-renders.
useReducer	État complexe (pattern Redux)	Formulaires complexes à champs multiples, machines à états finis.

useReducer — Gestion d'état complexe

ReactJS Avancé — Module approfondi

CONCEPTS CLÉS

useReducer s'impose face à useState lorsque la logique de mise à jour d'état implique une complexité significative ou des transitions conditionnelles multiples, car il externalise la gestion des effets de bord dans une fonction pure de réduction, facilitant ainsi la traçabilité et la testabilité via la séquence d'actions. Sous le capot, React optimise la réconciliation en comparant les états immuables issus du reducer, minimisant les rendus inutiles grâce à une granularité accrue du contrôle des mises à jour, ce qui est crucial dans les formulaires multi-étapes où chaque étape modifie un sous-ensemble cohérent de l'état global. L'adoption du pattern action/reducer favorise la séparation des préoccupations et permet l'intégration aisée de middleware pour la journalisation ou la gestion asynchrone, tout en limitant les closures excessives et en garantissant la stabilité des références lors des re-renders.

IMPLÉMENTATION

Exemple de code

```
import { useReducer } from 'react';

// Reducer function
const reducer = (state, action) => {
  switch (action.type) {
    case 'increment': return { count: state.count + 1 };
    case 'decrement': return { count: state.count - 1 };
    default: throw new Error();
  }
};
```

useReducer — Gestion d'état complexe

ReactJS Avancé — Module approfondi

CONCEPTS CLÉS

La stabilité des références dans les callbacks dispatch est cruciale pour éviter des re-renders superflus, surtout lorsqu'on passe dispatch en props à des composants enfants. Le pattern reducer facilite l'implémentation de fonctionnalités complexes comme le undo/redo en conservant un historique immuable des états, ce qui serait difficile à gérer avec useState. Lorsqu'on manipule des états profondément imbriqués, combiner useReducer avec des techniques d'optimisation telles que la mémoïsation des sélecteurs ou l'agrégation d'actions améliore significativement la performance et la maintenabilité de l'architecture globale.

IMPLÉMENTATION

Exemple de code

```
const Counter = () => {
  const [state, dispatch] = useReducer(reducer, { count: 0 });

  return (
    <>
      Count: {state.count}
      <button onClick={() => dispatch({ type: 'increment'
    })}></button>
      <button onClick={() => dispatch({ type: 'decrement' })}>-
    </button>
    </>
  );
};
```

useRef — Au-delà des références DOM

ReactJS Avancé — Module approfondi

CONCEPTS CLÉS

useRef crée un objet persistant dont la propriété `current` peut être mutée sans provoquer de réconciliation, car React ignore les mutations de ref lors du cycle de rendu, ce qui le différencie des états locaux. Cette immutabilité contrôlée permet de stocker des références stables à des valeurs ou instances externes, optimisant les performances en évitant des rerenders inutiles liés à des données non visuelles. Cependant, son usage doit être circonscrit aux cas où la synchronisation asynchrone ou le maintien d'un état mutable hors-cycle est requis, car l'abus peut entraîner des effets secondaires difficiles à tracer et compromettre la pureté des composants.

IMPLÉMENTATION

Exemple de code

```
import { useRef, useEffect } from 'react';

const Timer = () => {
  // 1. Référence DOM
  const inputRef = useRef<HTMLInputElement>(null);

  // 2. Stockage de valeur mutable (ne déclenche pas de re-render)
  const timerId = useRef<NodeJS.Timeout | null>(null);

  useEffect(() => {
    inputRef.current?.focus(); // Focus au montage
    timerId.current = setInterval(() => console.log('Tick'),
    1000);
    return () => clearInterval(timerId.current!);
  }, []);

  return <input ref={inputRef} placeholder="Auto-focus" />;
};
```


Maîtriser useEffect

Dépendances, cleanup et requêtes asynchrones

IMPLÉMENTATION CORRECTE

```
function UserProfile({ userId }) {  
  const [user, setUser] = useState(null);  
  
  useEffect(() => {  
    let cancelled = false;  
  
    async function fetchUser() {  
      const data = await api.getUser(userId);  
      if (!cancelled) setUser(data);  
    }  
  
    fetchUser();  
  
    // Cleanup : annuler si démonté  
    return () => { cancelled = true; };  
  }, [userId]); // Dépendance  
}
```

RÈGLES D'OR

Toujours nettoyer

Abonnements, timers, requêtes en cours → utiliser la fonction de retour (cleanup) pour éviter les fuites de mémoire.

Déclarer les dépendances

Tout ce qui est utilisé dans l'effet (props, state, fonctions) doit être inclus dans le tableau de dépendances.

[] = une seule fois

Un tableau vide `[]` signifie que l'effet ne s'exécutera qu'au montage du composant.

Pas de dépendance = chaque render

Si le tableau est omis, l'effet s'exécutera après chaque rendu du composant (souvent source de boucles infinies).

useEffect — Patterns et anti-patterns

ReactJS Avancé — Module approfondi

CONCEPTS CLÉS

Les dépendances omises dans le tableau de `useEffect` perturbent la réconciliation en provoquant des effets secondaires asynchrones non déterministes, compromettant la cohérence du DOM virtuel et entraînant des fuites de mémoire ou des boucles infinies de rendu. L'utilisation non conditionnelle de `setState` dans `useEffect` déclenche une re-réconciliation récursive, dégradant les performances et causant des rendus inutiles ; il est crucial de maîtriser la granularité des dépendances pour garantir l'immutabilité des états et optimiser la stabilité du cycle de vie des composants. Enfin, les requêtes `fetch` sans gestion explicite du nettoyage via `AbortController` ou équivalent peuvent laisser des mises à jour d'état post-mortem, introduisant des conditions de course et des erreurs silencieuses, d'où l'importance de concevoir des effets idempotents et réentrants pour garantir la robustesse des composants fonctionnels.

IMPLÉMENTATION

Exemple de code

```
// ❌ Anti-pattern : Boucle infinie ou dépendances manquantes
useEffect(() => {
  fetchData(query); // query manque dans le tableau
}, []);

// ✅ Pattern : Dépendances correctes et cleanup
useEffect(() => {
  const controller = new AbortController();

  fetchData(query, { signal: controller.signal })
    .then(setData);

  return () => controller.abort(); // Cleanup (annulation requête)
}, [query]); // Re-run si query change
```

Custom Hooks — Réutiliser la logique

Extraire et partager la logique d'état entre plusieurs composants

CRÉER USEFETCH<T>

```
function useFetch<T>(url: string) {  
  const [data, setData] = useState<T | null>(null);  
  const [loading, setLoading] = useState(true);  
  const [error, setError] = useState<Error | null>(null);  
  
  useEffect(() => {  
    fetch(url)  
      .then(res => res.json())  
      .then(setData)  
      .catch(setError)  
      .finally(() => setLoading(false));  
  }, [url]);  
  
  return { data, loading, error };  
}
```

UTILISATION DANS UN COMPOSANT

```
function ProductList() {  
  const { data, loading, error } = useFetch<Product[]>('/api/products');  
  
  if (loading) return <Spinner />;  
  if (error) return <ErrorMsg />;  
  
  return (  
    <ul>  
      {data?.map(p => (  
        <li key={p.id}>{p.name}</li>  
      ))}  
    </ul>  
  );  
}
```

Custom Hooks — Bibliothèque utilitaire

Exemples avancés de hooks React/TypeScript pour persistance, UI, fenêtre et interaction

Hooks de persistance et UI

```
import { useState, useEffect } from 'react';

function useLocalStorage<T>({ key: string, initialValue: T }): [T, React.Dispatch<React.SetStateAction<T>>] {
  const [storedValue, setStoredValue] = useState<T>()

  try {
    const item = window.localStorage.getItem(key)
    const initialValue = JSON.parse(item || '')
  } catch (error) {}

  return [initialValue, setStoredValue]
}

function useLocalStorage<T>({ key: string, initialValue: T }): [T, React.Dispatch<React.SetStateAction<T>>] {
  const [storedValue, setStoredValue] = useState<T>()

  try {
    const item = window.localStorage.getItem(key)
    const initialValue = JSON.parse(item || '')
  } catch (error) {}

  return [initialValue, setStoredValue]
}

useEffect(() => {
  try {
    window.localStorage.setItem(key, JSON.stringify(storedValue))
  } catch (error) {}
}, [key, storedValue])
```

Hooks de fenêtre et interaction

```
import { useState, useEffect } from 'react';

interface WindowSize {
  width: number
  height: number
}

function useWindowSize(): WindowSize {
  const [windowSize, setWindowSize] = useState<WindowSize>({
    width: window.innerWidth,
    height: window.innerHeight,
  })

  useEffect(() => {
    function handleResize() {
      setWindowSize({
        width: window.innerWidth,
        height: window.innerHeight,
      })
    }

    window.addEventListener('resize', handleResize)
    return () => window.removeEventListener('resize', handleResize)
  }, [])
```

Custom Hooks — Tests unitaires

ReactJS Avancé — Module approfondi

CONCEPTS CLÉS

La couverture unitaire des Custom Hooks nécessite d'isoler leurs effets de bord et états internes en exploitant le rendu simulé via `@testing-library/react-hooks`, qui expose le cycle de vie du hook dans un environnement contrôlé sans dépendre du DOM complet. Tester les transitions d'états asynchrones (loading, success, error) implique de maîtriser la synchronisation via des promises et la gestion fine de la réconciliation pour valider l'intégrité des mises à jour d'état immuables. Cette approche favorise la décomposition fonctionnelle et la réutilisabilité tout en limitant la complexité des tests d'intégration end-to-end, mais requiert une granularité rigoureuse pour éviter le couplage avec l'implémentation interne du hook.

IMPLÉMENTATION

Exemple de code

```
// Test d'un Custom Hook avec React Testing Library
import { renderHook, act } from '@testing-library/react-hooks';
import { useCounter } from './useCounter';

test('should increment counter', () => {
  const { result } = renderHook(() => useCounter(0));

  expect(result.current.count).toBe(0);

  // act() est requis pour les updates d'état dans les tests
  act(() => {
    result.current.increment();
  });

  expect(result.current.count).toBe(1);
});
```

Gestion des formulaires avec React Hook Form

ReactJS Avancé — Module approfondi

CONCEPTS CLÉS

React Hook Form exploite la gestion non contrôlée des inputs via des références directes au DOM, minimisant les re-renders en évitant le stockage localisé de l'état dans React, ce qui améliore significativement la performance pour des formulaires volumineux. L'intégration avec des schémas de validation déclaratifs comme Yup ou Zod permet une validation asynchrone et typée, déportant la logique métier hors du composant et favorisant la séparation des préoccupations. Contrairement à Formik, qui utilise un modèle contrôlé et entraîne des re-renders fréquents lors de chaque modification d'état, React Hook Form optimise la réconciliation en s'appuyant sur la gestion fine des refs et la mémorisation interne des callbacks, réduisant ainsi la charge sur le Virtual DOM.

IMPLÉMENTATION

Exemple de code

```
import { useForm } from 'react-hook-form';

const Form = () => {
  const { register, handleSubmit, formState: { errors } } =
    useForm();
  const onSubmit = data => console.log(data);
```

Gestion des formulaires avec React Hook Form

ReactJS Avancé — Module approfondi

CONCEPTS CLÉS

La gestion des erreurs et la configuration fine des règles de validation avec React Hook Form impactent directement l'expérience utilisateur et la maintenabilité, notamment à cause du pattern de validation déclarative. L'objet `errors` issu de `formState` offre une granularité précise pour l'affichage des messages, mais la gestion asynchrone des validations personnalisées peut engendrer des états intermédiaires non intuitifs, nécessitant parfois des optimisations via les modes de revalidation « `onBlur` » ou « `onSubmit` » pour éviter des validations trop agressives. Le pattern `{...register}` favorise un couplage faible avec les inputs, ce qui facilite les abstractions modulaires et l'intégration fluide d'UI libraries, mais demande une maîtrise rigoureuse de la propagation des refs, une complexité souvent sous-estimée dans des architectures React sophistiquées.

IMPLÉMENTATION

Exemple de code

```
return (
  <form onSubmit={handleSubmit(onSubmit)}>
    <input
      {...register("email", { required: true, pattern:
/^\\S+@\\S+$/i })}
    />
    {errors.email && <span>Email invalide</span>}

    <input
      type="password"
      {...register("password", { minLength: 8 })}
    />
    <input type="submit" />
  </form>
);
```

TP 3 — Contexts, Portals et Custom Hooks

Travaux pratiques · Module 3



Objectif : Implémenter des patterns avancés pour améliorer l'architecture de l'application

Étapes à suivre

1

Créer un AuthContext

Mettre en place un état global pour l'utilisateur connecté via l'API Context et créer un `AuthProvider`.

2

Implémenter une Modal avec Portal

Créer un composant Modal réutilisable qui s'affiche en dehors de la hiérarchie DOM principale.

```
ReactDOM.createPortal(child, container)
```

3

Développer useFetch<T>

Créer un Custom Hook générique pour les appels API, gérant les états `data`, `loading` et `error`.

Résultat attendu

Une application capable de gérer l'authentification globalement, d'afficher des modales propres (sans problèmes de z-index), et de récupérer des données via un hook réutilisable.

Livrable

Code source intégrant les 3 patterns, avec des composants propres et séparés.



Pièges fréquents :

Oublier d'envelopper l'application avec le

`AuthProvider`

. Mauvaise gestion des dépendances dans le

`useEffect`

du Custom Hook provoquant des boucles

infinies.

Module 4

Redux Avancé

Architecture Redux — Flux de données

ReactJS Avancé — Module approfondi

CONCEPTS CLÉS

Le flux de données Redux garantit une unidirectionnalité stricte en orchestrant les interactions via des actions immuables qui transitent par un dispatcher centralisé, lequel invoque des reducers purs pour produire un nouvel état sans effets de bord, facilitant ainsi la traçabilité et la prévisibilité des mutations d'état par réconciliation optimisée. Cette architecture découple la logique métier de la couche UI en externalisant le state management, ce qui améliore la maintenabilité et la testabilité à grande échelle, mais impose une surcharge cognitive et une verbosité accrue qu'il convient de compenser par des patterns comme la normalisation du state et la memoïsation sélective via des selectors. L'intégration avec React exploite la granularité du store et la propagation fine des changements pour minimiser les rerenders, favorisant un rendu performant dans des interfaces complexes où le flux unidirectionnel de React seul serait insuffisant pour gérer une logique d'état riche et évolutive.

IMPLÉMENTATION

Exemple de code

```
// Architecture Flux avec Redux
// 1. Action : Décrit ce qui s'est passé
const addTodo = (text) => ({
  type: 'todos/add',
  payload: text
});

// 2. Reducer : Calcule le nouvel état
const todosReducer = (state = [], action) => {
```

Architecture Redux — Flux de données

ReactJS Avancé — Module approfondi

CONCEPTS CLÉS

L'importance cruciale du pattern immutabilité dans les reducers, où la création systématique de nouveaux objets d'état via le spread operator garantit l'absence d'effets de bord et la fiabilité des comparaisons par référence. Toutefois, cette approche naïve peut engendrer des problèmes de performance sur des structures d'état volumineuses, justifiant l'adoption de bibliothèques comme Immer ou la normalisation approfondie du state pour limiter la profondeur des copies. Par ailleurs, la gestion fine des slices d'état par des reducers spécialisés favorise une architecture modulaire mais nécessite une composition rigoureuse avec combineReducers, sous peine de complexifier la maintenance et la scalabilité du store global. Enfin, la centralisation du store impose des choix architecturaux stratégiques liés à la synchronisation avec le middleware et les side effects, où des outils tels que Redux Saga ou Thunk deviennent incontournables pour orchestrer des flux asynchrones tout en conservant la pureté des reducers.

IMPLÉMENTATION

Exemple de code

```
switch (action.type) {
  case 'todos/add':
    return [...state, { text: action.payload, completed: false }];
  default:
    return state;
}

// 3. Store : Contient l'état global
```

Redux Toolkit — createSlice et Immer

La façon moderne et recommandée d'utiliser Redux

EXEMPLE : CARTSLICE

```
import { createSlice } from '@reduxjs/toolkit';

const cartSlice = createSlice({
  name: 'cart',
  initialState: { items: [], total: 0 },
  reducers: {
    addItem(state, action) {
      // Immer permet de "muter" l'état directement
      state.items.push(action.payload);
      state.total += action.payload.price;
    },
    clearCart(state) {
      state.items = [];
      state.total = 0;
    }
  }
});

export const { addItem, clearCart } = cartSlice.actions;
export default cartSlice.reducer;
```

AVANTAGES DE REDUX TOOLKIT

Moins de boilerplate

Plus besoin de définir manuellement les types d'actions, les action creators et les switch statements dans les reducers.

Immer intégré

Permet d'écrire une logique de mise à jour d'état qui semble muter les données, tout en conservant l'immutabilité requise par Redux.

Configuration simplifiée

`configureStore` configure automatiquement les Redux DevTools et les middlewares essentiels (comme thunk).

RTK Query inclus

Un outil puissant de data fetching et de mise en cache, éliminant le besoin d'écrire des thunks pour les requêtes API.

Redux Toolkit — configureStore et Middleware

Gestion avancée du store avec configuration typée et middlewares personnalisés

Configuration du Store

```
import { configureStore, TypedUseSelectorHook, useDispatch, useSelector } from '@reduxjs/toolkit';
import authReducer from './features/auth/authSlice';
import cartReducer from './features/cart/cartSlice';
import productsReducer from './features/products/productsSlice';
import { loggerMiddleware } from './middleware/loggerMiddleware';

export const store = configureStore({
  reducer: {
    auth: authReducer,
    cart: cartReducer,
    products: productsReducer,
  },
  middleware: (getDefaultMiddleware) =>
    getDefaultMiddleware().concat(loggerMiddleware),
});

export type RootState = ReturnType<typeof store.getState>;
export type AppDispatch = typeof store.dispatch;

export const useAppDispatch = () => useDispatch<AppDispatch>();
export const useAppSelector: TypedUseSelectorHook<RootState> = useSelector;
```

Middleware personnalisé

```
import { Middleware } from '@reduxjs/toolkit';

const loggerMiddleware: Middleware = (store) => (next) => (action) => {
  console.log('dispatching', action);
  const result = next(action);
  console.log('next state', store.getState());
  return result;
};
```

Ce middleware est un **pattern curried** : une fonction qui retourne une fonction, qui retourne une autre fonction.

store donne accès à **getState** et **dispatch**, **next** est la fonction suivante dans la chaîne des middlewares, et **action** est l'action Redux en cours de traitement.

Ce pattern permet d'intercepter, modifier ou logger les actions avant qu'elles n'atteignent le reducer.

Redux Toolkit — createAsyncThunk

ReactJS Avancé — Module approfondi

CONCEPTS CLÉS

`createAsyncThunk` encapsule la logique asynchrone en générant automatiquement des actions différenciées pour les états `pending`, `fulfilled` et `rejected`, facilitant ainsi la gestion explicite des effets secondaires dans les slices Redux sans middleware additionnel. Sous le capot, il exploite les Promises et la capacité de Redux Toolkit à gérer la réconciliation des états immutables, garantissant une mise à jour cohérente et prévisible du store tout en minimisant le boilerplate. Cette abstraction favorise un découplage clair entre la logique métier asynchrone et la couche de présentation, mais nécessite une attention particulière à la gestion des effets de bord et à la mémorisation des sélecteurs pour éviter les re-renderings inutiles.

IMPLÉMENTATION

Exemple de code

```
import { createAsyncThunk, createSlice } from '@reduxjs/toolkit';

// Thunk asynchrone
export const fetchUsers = createAsyncThunk('users/fetch', async ()
=> {
  const response = await fetch('/api/users');
  return response.json();
});
```

Redux Toolkit — createAsyncThunk

ReactJS Avancé — Module approfondi

CONCEPTS CLÉS

L'importance cruciale de gérer finement les différents états du cycle de vie d'une requête asynchrone via les extraReducers pour garantir une expérience utilisateur réactive et robuste. L'utilisation de builder.addCase permet une extension modulaire et typée du slice, mais nécessite une vigilance particulière sur la gestion immuable de l'état et la prévention des mutations directes, notamment dans les transitions pending, fulfilled et rejected. Par ailleurs, ce pattern s'intègre idéalement dans une architecture "feature-sliced" ou domain-driven où chaque slice encapsule ses propres effets secondaires, favorisant ainsi la scalabilité et la testabilité, tout en permettant d'envisager des optimisations avancées, telles que la normalisation des données et la mise en cache sélective via RTK Query pour réduire la surcharge réseau et les mises à jour redondantes.

IMPLÉMENTATION

Exemple de code

```
const usersSlice = createSlice({
  name: 'users',
  initialState: { data: [], status: 'idle' },
  reducers: {},
  extraReducers: (builder) => {
    builder
      .addCase(fetchUsers.pending, (state) => { state.status = 'loading'; })
      .addCase(fetchUsers.fulfilled, (state, action) => {
        state.status = 'succeeded';
        state.data = action.payload;
      });
  },
});
```

RTK Query — Data Fetching moderne

ReactJS Avancé — Module approfondi

CONCEPTS CLÉS

RTK Query encapsule la gestion des requêtes réseau via `createApi` en générant des hooks optimisés qui exploitent un cache normalisé pour minimiser les appels redondants et faciliter la réconciliation des données. Le `fetchBaseQuery` agit comme un middleware léger encapsulant `fetch`, permettant d'intercepter, transformer et invalider automatiquement les données en fonction des tags, garantissant ainsi la cohérence entre le cache local et l'état serveur. Cette abstraction favorise une architecture déclarative, réduit les effets de bord en isolant les side-effects, et optimise la performance grâce à la stratégie fine de polling et invalidation contrôlée.

IMPLÉMENTATION

Exemple de code

```
import { createApi, fetchBaseQuery } from
 '@reduxjs/toolkit/query/react';

// Configuration RTK Query
export const api = createApi({
  reducerPath: 'api',
  baseQuery: fetchBaseQuery({ baseUrl: '/api/' }),
  endpoints: (builder) => ({
    getUsers: builder.query<User[], void>({
      query: () => 'users',
    }),
  }),
});

export const { useGetUsersQuery } = api;

// Utilisation dans un composant
const { data, error, isLoading } = useGetUsersQuery();
```


Redux Persist — Persistence du state

Sauvegarder l'état global entre les sessions utilisateur

CONFIGURATION PERSISTCONFIG

```
import storage from 'redux-persist/lib/storage';
import { persistReducer } from 'redux-persist';
import authReducer from './authSlice';

const persistConfig = {
  key: 'auth',
  storage, // localStorage par défaut
  whitelist: ['token', 'user'], // Seulement ces champs
  // blacklist: ['error'] // Ou tout sauf ces champs
};

const persistedAuthReducer = persistReducer(
  persistConfig,
  authReducer
);
```

CONTRÔLE & CAS D'USAGE

Whitelist vs Blacklist

Utilisez `whitelist` pour définir explicitement les clés à sauvegarder, ou `blacklist` pour exclure les données éphémères (ex: erreurs, états de chargement).

Panier d'achat (E-commerce)

Conserver les articles ajoutés au panier même si l'utilisateur ferme l'onglet ou rafraîchit la page.

Préférences utilisateur

Sauvegarder le thème choisi (sombre/clair), la langue de l'interface ou les options d'affichage.

Authentification

Persiste le token JWT et les informations de profil pour maintenir la session active.

Redux — Selectors avec Reselect

ReactJS Avancé — Module approfondi

CONCEPTS CLÉS

Les selectors mémorisés créés via `createSelector` exploitent une cache interne basée sur les arguments d'entrée pour éviter le recalcul inutile lors de la réconciliation de l'arbre React, optimisant ainsi les performances en limitant les re-renders induits par des changements d'état identiques en valeur. Cette approche favorise la composition fonctionnelle, permettant d'agréger des sélecteurs simples en transformations plus complexes tout en préservant l'immutabilité et la pureté des fonctions, ce qui facilite la testabilité et la maintenabilité. Toutefois, il faut gérer avec soin la granularité de la mémorisation pour éviter des surcoûts mémoire ou des cache misses préjudiciables, notamment dans des états volumineux ou à structure profondément imbriquée.

IMPLÉMENTATION

Exemple de code

```
import { createSelector } from '@reduxjs/toolkit';

// Selectors basiques
const selectItems = state => state.shop.items;
const selectFilter = state => state.shop.filter;

// Selector mémorisé avec Reselect (inclus dans RTK)
export const selectFilteredItems = createSelector(
  [selectItems, selectFilter],
  (items, filter) => {
    // Ce calcul lourd n'est refait que si items ou filter
    // changent
    return items.filter(item => item.category === filter);
  }
);
```

React Query — Data Fetching et Cache

ReactJS Avancé — Module approfondi

CONCEPTS CLÉS

React Query exploite un cache asynchrone fondé sur une stratégie de réconciliation fine entre données locales et serveur, où `useQuery` orchestre la synchronisation en orchestrant les cycles de `staleTime` pour minimiser les requêtes redondantes via une politique d'actualisation optimisée. `useMutation` permet une gestion transactionnelle des modifications avec rollback automatique en cas d'échec, garantissant l'intégrité des données et une UX réactive sans blocage, tandis que l'invalidation explicite du cache provoque un refetch ciblé pour maintenir la cohérence des états dérivés. Comparé à RTK Query, React Query privilégie la séparation des préoccupations entre cache et logique métier en favorisant la composition fonctionnelle et la memoisation fine, ce qui réduit l'empreinte mémoire et améliore la scalabilité dans des architectures complexes à forte dynamique de données.

IMPLÉMENTATION

Exemple de code

```
import { useQuery } from '@tanstack/react-query';

const fetchUser = async () => {
  const res = await fetch('/api/user');
  return res.json();
};
```

React Query — Data Fetching et Cache

ReactJS Avancé — Module approfondi

CONCEPTS CLÉS

La configuration fine des options comme `staleTime`, `cacheTime`, et `retry` ouvre la voie à des optimisations cruciales pour la performance des applications à grande échelle, notamment en limitant les appels réseau inutiles tout en assurant une fraîcheur des données adaptée au contexte métier. Le pattern d'invalidation proactive du cache, combiné à des mécanismes de background fetching, permet de concilier réactivité UX et cohérence des états, mais nécessite une vigilance accrue face aux conditions de course et aux effets de bord lors des mises à jour concurrentes. Enfin, l'intégration de React Query dans une architecture modulaire s'appuie sur la séparation stricte des responsabilités entre couche de données et composants UI, favorisant la réutilisation des hooks personnalisés et facilitant le test unitaire isolé des effets secondaires liés au fetching et à la gestion du cache.

IMPLÉMENTATION

Exemple de code

```
const Profile = () => {  
  // React Query gère le cache, les retries, et le background  
  fetching  
  const { data, isLoading, error } = useQuery({  
    queryKey: ['user'],  
    queryFn: fetchUser,  
    staleTime: 5 * 60 * 1000, // Cache valide 5 minutes  
  });  
  
  if (isLoading) return <Spinner />;  
  if (error) return <ErrorAlert />;  
  return <div>{data.name}</div>;  
};
```

Zustand — Gestion d'état légère

ReactJS Avancé — Module approfondi

CONCEPTS CLÉS

Zustand implémente un store global minimaliste basé sur des hooks React, exploitant la closure pour encapsuler un état mutable tout en conservant une interface immuable, ce qui optimise la réconciliation en évitant les re-renders inutiles par une subscription granulaire aux sélecteurs. Contrairement à Redux, Zustand supprime la nécessité d'un boilerplate lourd et de middleware complexes, réduisant significativement la taille du bundle et la complexité cognitive tout en sacrifiant une gestion explicite des side-effects et un écosystème mature de devtools. Son usage recommandé privilégie les applications nécessitant une gestion d'état rapide et directe avec peu de logique métier asynchrone ou complexe, où la simplicité et la performance de la memoïsation par référence dominant.

IMPLÉMENTATION

Exemple de code

```
import create from 'zustand';

// Création du store Zustand (plus léger que Redux)
const useStore = create((set) => ({
  bears: 0,
  increasePopulation: () => set((state) => ({ bears: state.bears +
1 })),
  removeAllBears: () => set({ bears: 0 }),
}));
```

Zustand — Gestion d'état légère

ReactJS Avancé — Module approfondi

CONCEPTS CLÉS

L'importance de la granularité dans la sélection de l'état via les sélecteurs, conditionnant la précision des re-renders et la performance fine des composants. L'utilisation judicieuse de hooks comme `useStore` avec des sélecteurs isolés évite des re-renders globaux, mais expose aux pièges des closures capturant d'anciennes valeurs si les callbacks ne sont pas correctement mémorisés, notamment lors d'utilisation de fonctions dérivant de l'état. Architecturativement, Zustand favorise une composition modulaire des slices d'état, facilitant la séparation des responsabilités et la scalabilité sans complexifier la structure globale, tout en permettant d'intégrer aisément des middlewares sur mesure pour la gestion avancée des effets ou du logging, illustrant une flexibilité pragmatique face aux patterns plus rigides comme Redux.

IMPLÉMENTATION

Exemple de code

```
// Utilisation dans un composant
const BearCounter = () => {
  const bears = useStore((state) => state.bears);
  return <h1>{bears} bears</h1>;
};

const Controls = () => {
  const increase = useStore((state) => state.increasePopulation);
  return <button onClick={increase}>Add Bear</button>;
};
```

TP 4 — Redux Form, Persist et Middleware

Travaux pratiques · Module 4



Objectif : Convertir le formulaire en Redux Form, persister le slice auth et créer un middleware logger.

Étapes à suivre

1

Installer les dépendances

```
npm install redux-persist
```

2

Configurer le store avec Persist

Ajouter `persistReducer` et `persistStore` pour sauvegarder le slice d'authentification.

3

Créer le middleware logger

Intercepter chaque action dispatchée et l'afficher dans la console avec l'état précédent et suivant.

4

Intégrer Redux Form

Remplacer l'état local du formulaire de connexion par un état géré via Redux.

5

Tester la persistance

Se connecter, recharger la page et vérifier que l'utilisateur reste authentifié.

Résultat attendu

Formulaire géré par Redux, état d'authentification conservé après rechargement (localStorage), et actions logguées dans la console.

Livable

Code source mis à jour avec le store configuré, le middleware actif et l'application enveloppée par `PersistGate`.



Pièges fréquents :

Oublier d'envelopper l'application avec

`<PersistGate>`

. Erreur de sérialisation avec Redux Toolkit (ignorer les actions persistantes dans le middleware).

Module 5

Optimisation des Performances

Profiler React — Identifier les goulots

Utilisation avancée du Profiler dans React DevTools et en code

Utiliser le Profiler

4 étapes pour analyser les performances avec React DevTools :

1. Ouvrir React DevTools et sélectionner l'onglet **Profiler**.
2. Cliquer sur le bouton **Record** pour démarrer l'enregistrement.
3. Effectuer l'interaction utilisateur à analyser dans l'application.
4. Cliquer sur **Stop** pour arrêter l'enregistrement et visualiser les résultats.

Description du flamegraph :

Chaque barre représente un composant React. La largeur de la barre correspond au temps de rendu, et la couleur indique la fréquence de rendu : gris signifie pas de re-rendu, tandis que les couleurs chaudes indiquent un re-rendu fréquent.

Métriques clés :

- **Render duration** : durée du rendu du composant.
- **Why did this render?** : explication des causes de re-rendu.

Utiliser le composant Profiler

```
<Profiler id= "ProductList" onRender= { onRenderCallback } >
  /* Composants enfants à profiler */
</Profiler>

function onRenderCallback(duration: number, commitTime: number, update: number) {
  // Exemple : log des mesures pour analyse
  console.log (
    `Profiling ${id} during ${phase} phase` ,
    { actualDuration, baseDuration, startTime, commitTime }
  );
}
```

Cas d'usage : mesurer l'impact d'une optimisation en production en activant le Profiler avec **sampling**. Cela permet d'obtenir des données précises sans dégrader les performances globales.

Code Splitting avec React.lazy et Suspense

Réduire le bundle initial de 40 à 70% pour un chargement plus rapide

IMPLÉMENTATION SUR LES ROUTES

```
import React, { Suspense, lazy } from 'react';
import { BrowserRouter, Routes, Route } from 'react-router-dom';

// Chargement asynchrone des composants
const Home = lazy(() => import('./pages/Home'));
const Dashboard = lazy(() => import('./pages/Dashboard'));

function App() {
  return (
    <BrowserRouter>
      <!-- Fallback UI pendant le chargement -->
      <Suspense fallback={<div>Chargement...</div>}>
        <Routes>
          <Route path="/" element={<Home />} />
          <Route path="/dashboard" element={<Dashboard />} />
        </Routes>
      </Suspense>
    </BrowserRouter>
  );
}
```

BONNES PRATIQUES

Granularité par route

C'est le point de départ idéal. Chaque page devient un fichier séparé (chunk) qui n'est téléchargé que lorsque l'utilisateur navigue vers cette route.

Composants lourds conditionnels

Utilisez `React.lazy` pour les composants très lourds (graphiques D3, éditeurs de texte riche, modales complexes) qui ne sont pas visibles immédiatement.

Analyser le bundle

Utilisez des outils comme `source-map-explorer` ou `rollup-plugin-visualizer` (avec Vite) pour identifier les dépendances qui alourdissent votre application.

Fallback UI soigné

Fournissez un composant de chargement élégant (Skeleton screen ou Spinner) dans la prop `fallback` de `Suspense` pour éviter les sauts visuels.

Virtualisation des listes — react-window

ReactJS Avancé — Module approfondi

CONCEPTS CLÉS

React-window implémente la virtualisation des listes en ne montant dans le DOM que les éléments visibles et un buffer adjacent, réduisant drastiquement le coût de rendu et la consommation mémoire lors du traitement de grandes collections. FixedSizeList repose sur une hauteur d'item constante permettant un calcul direct des offsets via une arithmétique simple, optimisant la réconciliation et le positionnement, tandis que VariableSizeList gère des tailles dynamiques par une table de lookup et un recalcul partiel, introduisant un compromis en complexité et coût de mise à jour. Ce pattern exploite l'immuabilité des données et la mémoïsation pour minimiser les rerenders, mais nécessite une gestion fine des clés et des callbacks pour éviter les effets de bord liés aux closures et garantir la fluidité de l'interface.

IMPLÉMENTATION

Exemple de code

```
import { FixedSizeList as List } from 'react-window';

// Rendu virtuel pour des listes massives (ex: 10 000 éléments)
const Row = ({ index, style }) => (
  <div style={style}>Row {index}</div>
);
```

Virtualisation des listes — react-window

ReactJS Avancé — Module approfondi

CONCEPTS CLÉS

La gestion des callbacks et des styles dans react-window révèle des subtilités cruciales pour la performance et la maintenabilité. Par exemple, l'utilisation d'une fonction Row pure et mémorisée évite des rerenders inutiles, mais attention à la stabilité des props style générées dynamiquement, source fréquente de reflow coûteux. Architecturalement, intégrer react-window pousse à repenser la granularité des composants et l'état local versus global, notamment quand des interactions complexes (sélections, filtres) doivent être synchronisées sans forcer le remontage complet de la liste. Enfin, comparer avec des alternatives comme react-virtualized ou react-virtuoso souligne les compromis entre flexibilité (gestion fine des tailles dynamiques, scroll infini) et simplicité d'usage, orientant le choix vers des solutions adaptées au contexte métier et à la taille des données manipulées.

IMPLÉMENTATION

Exemple de code

```
const VirtualizedList = () => (  
  <List  
    height={400}  
    itemCount={10000}  
    itemSize={35}  
    width={300}  
  >  
    {Row}  
  </List>  
)  
// Seuls les éléments visibles sont présents dans le DOM !
```

Web Workers avec React

ReactJS Avancé — Module approfondi

CONCEPTS CLÉS

L'intégration de Web Workers dans une application React permet d'externaliser les traitements CPU-bound vers un thread dédié, évitant ainsi le blocage du thread principal et améliorant la réactivité de l'UI grâce à la non-bloquance du moteur JavaScript unique. Comlink facilite la sérialisation transparente des appels et des réponses via la `postMessage` API, en masquant la complexité de la communication asynchrone inter-thread par une abstraction de proxy, tout en minimisant le coût de transfert des données grâce à la clonabilité structurée. L'architecture doit cependant gérer explicitement l'immuabilité des messages échangés pour éviter les effets de bord, et optimiser la granularité des tâches déléguées afin de réduire la surcharge induite par la latence de communication entre threads.

IMPLÉMENTATION

Exemple de code

```
// Main thread (React)
const worker = new Worker(new URL('./worker.js',
import.meta.url));

worker.postMessage({ data: heavyData });
worker.onmessage = (e) => {
  setResult(e.data); // Met à jour l'UI quand le calcul est fini
};

// worker.js (Thread séparé)
self.onmessage = (e) => {
  // Calcul intensif qui ne bloque pas l'UI React
  const result = heavyComputation(e.data);
  self.postMessage(result);
};
```

useMemo, useCallback et React.memo

Éviter les re-renders inutiles et optimiser les performances

React.memo

Optimise un composant entier. Il empêche le re-render si les props reçues n'ont pas changé (comparaison superficielle).

```
const MyComponent = React.memo(  
  function MyComponent(props) {  
    return <div>{props.text}</div>;  
  }  
);
```

useMemo

Mémoïse une valeur calculée. Utile pour éviter de refaire des calculs coûteux (tri, filtrage) à chaque rendu.

```
const memoizedValue = useMemo(  
  () => computeExpensive(a, b),  
  [a, b] // Dépendances  
);
```

useCallback

Mémoïse une référence de fonction. Indispensable lorsqu'on passe une fonction en prop à un composant enfant optimisé avec React.memo.

```
const memoizedCallback = useCallback(  
  () => { doSomething(a, b); },  
  [a, b] // Dépendances  
);
```

Règles d'or de l'optimisation

1. **Profiler d'abord** : Utilisez les React DevTools (onglet Profiler) pour identifier les vrais goulots d'étranglement avant d'optimiser.
2. **Ne pas sur-optimiser** : L'utilisation de ces hooks a un coût (mémoire et calcul des dépendances). Ne les utilisez pas par défaut sur chaque variable ou fonction.

Optimisation du rendu — Récapitulatif

ReactJS Avancé — Module approfondi

CONCEPTS CLÉS

La maîtrise du rendu optimal en React repose sur une compréhension fine du processus de réconciliation, où `React.memo` évite la re-rendu de composants purs en s'appuyant sur la comparaison superficielle des props pour minimiser les calculs inutiles. L'utilisation combinée de `useMemo` et `useCallback` exploite la mémoïsation basée sur des closures pour stabiliser les références fonctionnelles et les valeurs calculées, réduisant ainsi les effets secondaires et les recalculs dans l'arbre de composants, mais nécessite une gestion rigoureuse des dépendances pour éviter les bugs liés à la stale closure. Parallèlement, la virtualisation améliore la performance de rendu sur de larges listes en limitant le DOM au viewport, tandis que le code splitting dissocie les bundles pour optimiser le temps de chargement initial, chacune de ces techniques engageant un compromis entre complexité de maintenance, latence perçue et empreinte mémoire.

IMPLÉMENTATION

Exemple de code

```
// Récapitulatif des optimisations React
import { memo, useMemo, useCallback } from 'react';

// 1. memo : Évite le re-render si les props sont identiques
const Child = memo(({ onClick, data }) => {
  return <button onClick={onClick}>{data.title}</button>;
});

const Parent = () => {
  // 2. useMemo : Mémoïse une valeur complexe
  const data = useMemo(() => ({ title: "Hello" }), []);

  // 3. useCallback : Mémoïse une référence de fonction
  const handleClick = useCallback(() => console.log('Click'), []);

  return <Child data={data} onClick={handleClick} />;
};
```

Monitoring et observabilité

Surveillance avancée des applications React pour fiabilité et performance

Sentry — Suivi des erreurs

// main.tsx - Initialisation de Sentry

```
import * as Sentry from '@sentry/react' ;
import { Integrations } from '@sentry/tracing' ;
import React from 'react' ;
import ReactDOM from 'react-dom/client' ;
import App from './App'
```

```
Sentry.
({
  init
, dsn: 'https://examplePublicKey@o0.ingest.sentry.io/0'
, environment: 'production'
, integrations: [new Integrations.BrowserTracing()],
  tracesSampleRate: 0.2,
});
```

// Wrapper Sentry pour capturer erreurs React

```
const Root = () => (
  <Sentry.ErrorBoundary fallback={ 'Une erreur est survenue.' }>
    <App />
  </Sentry.ErrorBoundary>
);
```

```
const root = ReactDOM.
  createRoot( document.getElementById( 'root' )
  {<Root />});
```

Core Web Vitals et Lighthouse

LCP (Largest Contentful Paint) < 2.5s

FID/INP (First Input Delay / Interaction to Next Paint) < 100ms

CLS (Cumulative Layout Shift) < 0.1

Lighthouse CI intégré dans le pipeline CI/CD via `lighthouse-ci/action`
web-vitals library pour mesurer en production avec `reportWebVitals`
Google Search Console pour monitoring continu des Core Web Vitals

Objectif : score Lighthouse > 90 en production

TP 5 — Optimisation : Code Splitting

Travaux pratiques · Module 5



Objectif : Réduire le bundle initial d'au moins 30% en appliquant le Code Splitting et la mémoïsation.

Étapes à suivre

1

Analyser le bundle actuel

Utiliser un outil comme `rollup-plugin-visualizer` pour identifier les plus gros modules.

2

Appliquer `React.lazy`

Remplacer les imports statiques des pages principales par des imports dynamiques.

```
const Dashboard = lazy(() => import('./Dashboard'))
```

3

Ajouter `Suspense`

Envelopper les routes avec `<Suspense fallback={...}>` pour gérer l'état de chargement.

4

Utiliser `React.memo`

Identifier un composant enfant lourd et l'envelopper avec `React.memo` pour éviter les re-renders.

5

Mesurer l'impact

Générer un nouveau rapport d'analyse et comparer la taille du bundle initial.

Résultat attendu

Le fichier JavaScript principal (`main.js`) est significativement plus petit. Les autres pages sont chargées à la demande (chunks séparés).

Livable

Captures d'écran du bundle analyzer (Avant / Après) montrant la réduction de taille.



Pièges fréquents :

Oublier le composant `Suspense` (provoque une erreur React). Sur-optimiser avec `React.memo` sur des composants simples (coût de comparaison supérieur au gain).

Module 6

Animations & Transitions

React Transition Group

Animer l'apparition et la disparition des composants dans le DOM

COMPOSANT REACT

```
import { CSSTransition } from 'react-transition-group';

function AlertMessage({ show, message }) {
  return (
    <CSSTransition
      in={show}
      timeout={300}
      classNames="fade"
      unmountOnExit
    >
      <div className="alert">
        {message}
      </div>
    </CSSTransition>
  );
}
```

Contrairement au CSS pur, **RTG** permet de retarder le démontage du composant (`unmountOnExit`) le temps que l'animation se termine.

CLASSES CSS GÉNÉRÉES

.fade-enter

État initial avant l'apparition.

.fade-enter-active

État final de l'apparition (transition).

.fade-exit

État initial avant la disparition.

.fade-exit-active

État final de la disparition (transition).

```
/* Fichier CSS */
.fade-enter {
  opacity: 0;
  transform: translateY(-10px);
}
.fade-enter-active {
  opacity: 1;
  transform: translateY(0);
  transition: opacity 300ms, transform 300ms;
}
.fade-exit { opacity: 1; }
.fade-exit-active {
  opacity: 0;
  transition: opacity 300ms;
}
```

Framer Motion — Animations déclaratives

ReactJS Avancé — Module approfondi

CONCEPTS CLÉS

Framer Motion exploite un système déclaratif basé sur des variants, permettant de définir des états d'animation sous forme d'objets immuables qui favorisent la réconciliation optimisée du DOM virtuel par React. Les props `initial`, `animate` et `exit` orchestrent respectivement les phases de montage, transition et démontage en synchronisant la gestion des frames via la boucle d'animation `requestAnimationFrame`, garantissant ainsi des transitions fluides et performantes. `AnimatePresence` gère le cycle de vie des composants conditionnels en maintenant leur présence dans le DOM jusqu'à la fin des animations de sortie, évitant les suppressions prématurées et facilitant un contrôle précis des séquences d'animation dans une architecture déclarative.

IMPLÉMENTATION

Exemple de code

```
import { motion } from 'framer-motion';

const AnimatedBox = () => (
  <motion.div
    initial={{ opacity: 0, scale: 0.5 }}
    animate={{ opacity: 1, scale: 1 }}
    transition={{ duration: 0.5 }}
    whileHover={{ scale: 1.1 }}
    whileTap={{ scale: 0.9 }}
  >
    Composant Animé
  </motion.div>
);
// Framer Motion gère automatiquement les interpolations CSS
```

Framer Motion — Gestures et Layout

ReactJS Avancé — Module approfondi

CONCEPTS CLÉS

La gestion des gestes avec Framer Motion exploite un système d'événements bas-niveau couplé à la physique déclarative, permettant de contrôler précisément les interactions via drag et dragConstraints qui limitent le déplacement dans un espace contraint tout en assurant une réconciliation fluide des états. L'animation de layout repose sur la détection des changements de position et de taille via la prop layout, qui déclenche une interpolation optimisée entre les frames pour minimiser les recalculs du DOM et réduire le layout thrashing, améliorant ainsi la performance perçue. L'utilisation avancée de layoutId permet de coordonner des animations entre des composants distincts en synchronisant leurs identifiants de layout, exploitant une logique de morphing et de transition partagée qui facilite la gestion des états complexes sans recourir à des side effects explicites ni à la duplication de logique.

IMPLÉMENTATION

Exemple de code

```
import { motion, AnimatePresence } from 'framer-motion';

// Animation de montage / démontage
const Modal = ({ isVisible }) => (
  <AnimatePresence>
```

Framer Motion — Gestures et Layout

ReactJS Avancé — Module approfondi

CONCEPTS CLÉS

Le pattern `AnimatePresence` joue un rôle crucial dans la gestion asynchrone des cycles de vie des composants animés. En optimisant la synchronisation des états d'entrée et de sortie via les props `initial`, `animate` et `exit`, Framer Motion évite les glitches fréquents liés au démontage prématuré, tout en permettant des animations orchestrées complexes sans recourir à des callbacks manuels. La composition avec `layoutId` et `AnimatePresence` ouvre la voie à une architecture déclarative robuste, où les transitions entre états UI se font naturellement, favorisant un découplage clair entre logique métier et gestion fine des animations, essentiel pour la maintenabilité et la scalabilité des applications React de grande envergure.

IMPLÉMENTATION

Exemple de code

```
{isVisible && (  
  <motion.div  
    initial={{ opacity: 0, y: -50 }}  
    animate={{ opacity: 1, y: 0 }}  
    exit={{ opacity: 0, y: 50 }} // Animation à la destruction  
  !  
  >  
    Contenu de la modale  
  </motion.div>  
  )}  
</AnimatePresence>  
);
```

Comparatif des librairies d'animation

Choisir le bon outil en fonction du besoin et de la complexité

Librairie	Complexité	Performance	Cas d'usage principal
CSS Transitions	Faible	Excellente	Animations simples (hover, focus, changements d'état basiques).
React Transition Group	Moyenne	Bonne	Apparition/disparition de composants du DOM, transitions de routes.
Framer Motion	Moyenne	Excellente	Animations déclaratives complexes, drag & drop, gestes, layout animations.
React Spring	Élevée	Excellente	Animations basées sur la physique (ressorts) pour un rendu très naturel.
GSAP	Élevée	Excellente	Séquences d'animation très complexes, animations liées au scroll (ScrollTrigger).

CSS-in-JS — Styled Components

ReactJS Avancé — Module approfondi

CONCEPTS CLÉS

Styled-components implémente un CSS-in-JS via des tagged template literals qui génèrent des classes CSS uniques à la compilation, assurant l'isolation des styles par composant tout en exploitant la puissance des closures pour injecter dynamiquement des props dans les règles CSS. Ce paradigme facilite la composition et le theming via ThemeProvider, qui propage un contexte immuable optimisant la réconciliation des styles lors des mises à jour, mais induit un surcoût en runtime lié à la génération et l'injection des feuilles de style, impactant la performance dans les applications à grande échelle. L'adoption de techniques telles que la memoïsation des fonctions de styles et la séparation explicite des styles statiques et dynamiques est cruciale pour minimiser les recalculs et préserver la maintenabilité dans des architectures complexes.

IMPLÉMENTATION

Exemple de code

```
import styled from 'styled-components';

// Création d'un composant stylisé avec props dynamiques
const Button = styled.button`
  background: ${props => props.primary ? '#2B6CB0' : 'white'};
  color: ${props => props.primary ? 'white' : '#2B6CB0'};
  padding: 10px 20px;
  border: 2px solid #2B6CB0;
  border-radius: 4px;
```


CSS-in-JS — Styled Components

ReactJS Avancé — Module approfondi

CONCEPTS CLÉS

Au-delà de l'écriture déclarative, ce code illustre la gestion fine des pseudo-classes CSS en CSS-in-JS, en particulier l'intégration native de l'état `hover` qui évite les manipulations DOM supplémentaires. Toutefois, la concaténation dynamique des styles via les props peut entraîner une explosion combinatoire des classes générées, posant un challenge de performance et de cache CSS, notamment lorsqu'on multiplie les variantes et états. Pour pallier ce problème, l'adoption de patterns comme le style interpolation conditionnel ou la composition par extensions de styled components, couplée à une stratégie stricte de séparation des styles statiques et dynamiques, devient essentielle afin de maintenir un bundle CSS raisonnable tout en conservant la flexibilité expressive.

IMPLÉMENTATION

Exemple de code

```
cursor: pointer;
transition: all 0.2s;

&:hover {
  background: #1A365D;
  color: white;
}
`;

// Utilisation
<Button primary>Envoyer</Button>
```

TP 6 — Transitions d'écrans fluides

Travaux pratiques · Module 6



Objectif : Ajouter des transitions fluides (fade-in / fade-out) entre toutes les pages de l'application.

Étapes à suivre

1

Installer React Transition Group

```
npm install react-transition-group
```

2

Créer le composant PageTransition

Créer un composant wrapper utilisant `CSSTransition`.

3

Entourer les routes

Envelopper le composant `<Routes>` avec `<TransitionGroup>`.

4

Définir les classes CSS

Créer les classes `.fade-enter`, `.fade-enter-active`, etc. dans le fichier CSS global.

5

Animer les listes (Bonus)

Appliquer le même principe pour animer l'ajout/suppression d'éléments dans la liste de produits.

Résultat attendu

Lors de la navigation via le menu, l'ancienne page disparaît en fondu pendant que la nouvelle apparaît, sans saut visuel brusque.

Livable

Code source mis à jour avec le routage animé et les styles CSS correspondants.



Pièges fréquents:

Oublier de passer `location.key` à `CSSTransition` (l'animation ne se déclenchera pas). Problèmes de superposition si les pages ne sont pas en fondu pendant la transition.

Module 7

Internationalisation (i18n)

Intégrer react-i18next

Configuration globale et utilisation du hook useTranslation

1. CONFIGURATION (I18N.TS)

```
import i18n from 'i18next';
import { initReactI18next } from 'react-i18next';

// Ressources de traduction (souvent dans des fichiers JSON)
const resources = {
  en: { translation: { welcome: "Welcome to React" } },
  fr: { translation: { welcome: "Bienvenue sur React" } }
};

i18n
  .use(initReactI18next)
  .init({
    resources,
    lng: "fr", // Langue par défaut
    fallbackLng: "en",
    interpolation: {
      escapeValue: false // React protège déjà contre XSS
    }
  });

export default i18n;
```

2. UTILISATION DANS UN COMPOSANT

```
import { useTranslation } from 'react-i18next';

function Header() {
  // t: fonction de traduction, i18n: instance globale
  const { t, i18n } = useTranslation();

  const changeLang = (lang: string) => {
    i18n.changeLanguage(lang);
  };

  return (
    <header>
      <h1>{t('welcome')}</h1>
      <button onClick={() => changeLang('fr')}>FR</button>
      <button onClick={() => changeLang('en')}>EN</button>
    </header>
  );
}
```

Changement dynamique : L'appel à `i18n.changeLanguage()` déclenche automatiquement un re-render de tous les composants utilisant `useTranslation()`, mettant à jour l'interface instantanément.

i18n — Pluralisation et interpolation

ReactJS Avancé — Module approfondi

CONCEPTS CLÉS

La gestion de la pluralisation avec i18next repose sur un algorithme de sélection des règles plurales définies par la norme CLDR, intégrant des fonctions conditionnelles spécifiques à chaque langue pour assurer une réconciliation précise entre le nombre et la forme linguistique. L'interpolation sécurisée des variables utilise des templates littéraux couplés à une évaluation différée, garantissant l'immuabilité des chaînes traduites tout en évitant les injections de code et en maintenant la pureté des fonctions de traduction. Le formatage localisé des dates et nombres exploite l'API Intl native, permettant une abstraction performante et cohérente des conventions régionales, ce qui favorise la séparation des responsabilités entre la logique métier et la présentation tout en optimisant la mémoire des formats les plus utilisés.

IMPLÉMENTATION

Exemple de code

```
// react-i18next : Interpolation et Pluralisation
import { useTranslation } from 'react-i18next';

// JSON de traduction (fr.json) :
// "messages": "Vous avez {{count}} message",
// "messages_plural": "Vous avez {{count}} messages"

const Inbox = ({ count }) => {
  const { t } = useTranslation();

  return (
    <div>
      {/* i18next gère automatiquement le pluriel basé sur la
      variable 'count' */}
      <h1>{t('messages', { count })}</h1>
    </div>
  );
};
```

i18n — Chargement à la demande (Lazy Loading)

ReactJS Avancé — Module approfondi

CONCEPTS CLÉS

Le chargement à la demande des fichiers de traduction via i18next-http-backend exploite la dynamique d'import pour réduire la charge initiale en réseau et mémoire, en récupérant uniquement les ressources linguistiques nécessaires au contexte utilisateur. Cette approche optimise la performance perçue et la scalabilité, mais nécessite une gestion fine du cache et de la synchronisation asynchrone pour prévenir les artefacts d'affichage dus à la latence réseau. Par ailleurs, la découpe granulaire des bundles de traduction doit être conçue en accord avec la granularité fonctionnelle de l'application afin de maximiser l'efficacité du lazy loading tout en maintenant la cohérence UX.

IMPLÉMENTATION

Exemple de code

```
import i18n from 'i18next';
import Backend from 'i18next-http-backend';
import { initReactI18next } from 'react-i18next';
```

i18n — Chargement à la demande (Lazy Loading)

ReactJS Avancé — Module approfondi

CONCEPTS CLÉS

Le `fallbackLng` est crucial pour garantir une expérience utilisateur fluide en cas d'échec de chargement ou de langue non supportée, évitant ainsi les écrans vides. L'utilisation de `debug` à `false` prévient la pollution des logs en production, mais nécessite un monitoring spécifique pour détecter silencieusement les erreurs de chargement asynchrones. Cette configuration ouvre la voie à des optimisations avancées, comme le préchargement conditionnel basé sur le comportement utilisateur, et l'intégration avec des solutions de cache côté client (IndexedDB, Service Workers) pour minimiser les requêtes, accélérer la réactivité, tout en conservant la maintenabilité et la cohérence architecturale des bundles i18n.

IMPLÉMENTATION

Exemple de code

```
// Configuration avec Lazy Loading des traductions
i18n
  .use(Backend) // Charge les fichiers depuis /public/locales/
  .use(initReactI18next)
  .init({
    fallbackLng: 'en',
    debug: false,
    interpolation: { escapeValue: false },
    backend: {
      loadPath: '/locales/{{lng}}/{{ns}}.json',
    }
  });

export default i18n;
```

TP 7 — Internationalisation complète

Travaux pratiques · Module 7



Objectif : Rendre l'application bilingue (Français / Anglais) avec changement de langue à la volée.

Étapes à suivre

1

Installer i18next

```
npm install i18next react-i18next i18next-browser-languagedetector
```

2

Créer les fichiers JSON

Créer `fr.json` et `en.json` contenant les paires clé-valeur pour toutes les chaînes de l'application.

3

Extraire les textes

Remplacer tous les textes en dur dans les composants par l'appel au hook `useTranslation()`.

4

Créer un LanguageSwitcher

Ajouter un menu déroulant dans la navbar pour permettre à l'utilisateur de changer de langue.

5

Persister la langue

Configurer le plugin `LanguageDetector` pour sauvegarder le choix dans le `localStorage`.

Résultat attendu

L'application s'affiche dans la langue du navigateur par défaut. L'utilisateur peut changer la langue via le menu, et ce choix est conservé après rechargement.

Livable

Code source avec configuration i18n complète, fichiers de traduction séparés et composants mis à jour.



Pièges fréquents :

Oublier d'importer le fichier de configuration `i18n.ts` dans `main.tsx`. Utiliser des clés de traduction qui n'existent pas dans les fichiers JSON (affiche la clé brute).

Ressources & Documentation officielle

ReactJS Avancé — Module approfondi

CONCEPTS CLÉS

La documentation officielle de React (react.dev) détaille le processus de réconciliation et l'optimisation du Virtual DOM, expliquant comment minimiser les opérations coûteuses du DOM réel par une comparaison efficace des arbres virtuels. Les ressources sur Redux Toolkit exposent la gestion immuable et prévisible de l'état via des slices, facilitant la normalisation et la mémorisation des sélecteurs pour éviter des rendus inutiles. Les guides Next.js approfondissent le rendu hybride SSR/SSG, soulignant les compromis entre performance, SEO et complexité côté serveur, tandis que le TypeScript Handbook explicite les systèmes de types avancés permettant d'assurer la robustesse et la maintenabilité à grande échelle.

IMPLÉMENTATION

Exemple de code

```
// Ressources Officielles et Recommandées
// 1. React Docs (Nouveau site) : react.dev
// 2. Redux Toolkit : redux-toolkit.js.org
// 3. React Query : tanstack.com/query
// 4. Next.js : nextjs.org/docs
// Utilitaires indispensables :
// - ESLint & Prettier (Qualité)
// - Vite (Build tool ultra-rapide)
// - Vitest (Tests unitaires rapides)
// - Playwright (Tests E2E)
```

Glossaire des termes clés

ReactJS Avancé — Module approfondi

CONCEPTS CLÉS

Les Higher-Order Components (HOC) exploitent le pattern de fonction d'ordre supérieur pour injecter dynamiquement des comportements en manipulant la composition de composants React, favorisant ainsi la réutilisabilité tout en introduisant potentiellement des complexités dans la gestion des props et des refs. Les Custom Hooks encapsulent des logiques d'état et d'effets via les closures JavaScript, garantissant l'immutabilité et la séparation des préoccupations, ce qui optimise la testabilité et la maintenance des effets secondaires liés au cycle de vie. La mémoïsation, via React.memo ou useMemo, réduit les recalculs redondants en exploitant la comparaison référentielle, mais nécessite une granularité fine pour éviter des surcoûts inutiles lors de la réconciliation. Le code splitting, implémenté par dynamic import, optimise le chargement initial en fractionnant les bundles, tout en complexifiant la gestion des dépendances et le rendu asynchrone. Le Server-Side Rendering (SSR) et le Static Site Generation (SSG) améliorent le SEO et le Time-to-Interactive en pré-rendant le DOM côté serveur, mais SSR induit une charge serveur accrue et des coûts réseau. L'Incremental Static Regeneration (ISR) combine ces approches en régénérant les pages statiques de manière incrémentale, équilibrant ainsi actualisation des données et performance. L'hydratation synchronise le DOM statique

IMPLÉMENTATION

Exemple de code

```
// Glossaire des termes clés React
const glossary = {
  JSX: "Extension syntaxique JavaScript pour écrire des éléments UI",
  VirtualDOM: "Représentation en mémoire du DOM pour optimiser les rendus",
  Hook: "Fonction permettant de se brancher sur l'état et le cycle de vie",
  HOC: "Higher Order Component - Fonction qui renvoie un nouveau composant",
  Thunk: "Middleware Redux pour gérer la logique asynchrone",
  SSR: "Server-Side Rendering - Rendu HTML généré sur le serveur",
  Hydration: "Processus d'attachement des event listeners React au HTML SSR"
};
```

Checklist de qualité — Avant de livrer

ReactJS Avancé — Module approfondi

CONCEPTS CLÉS

La validation du code s'appuie sur l'activation du mode strict de TypeScript pour garantir un typage statique exhaustif, minimisant ainsi les erreurs runtime et facilitant la maintenance par inférence précise des types complexes. La couverture de tests unitaires et d'intégration supérieure à 70 % assure une robustesse fonctionnelle, tandis que la suppression des appels `console.log` prévient les effets secondaires non désirés dans la sortie standard, améliorant la lisibilité et la sécurité en production. Sur le plan des performances, l'analyse fine du bundle détecte les dépendances redondantes et favorise la fragmentation granulaire via le lazy loading, limitant la charge initiale et optimisant la réconciliation React en réduisant les re-renders inutiles par une gestion rigoureuse de l'immuabilité et de la mise en cache des composants.

IMPLÉMENTATION

Exemple de code

```
// Checklist de Qualité avant Mise en Production
const qualityChecklist = [
  "✓ Aucun warning ESLint ou TypeScript restant",
  "✓ Pas de console.log() oubliés en production",
  "✓ Les clés (keys) dans les listes sont uniques (pas d'index)",
  "✓ Les dépendances useEffect sont complètes",
  "✓ Le Code Splitting (React.lazy) est implémenté sur les grosses routes",
  "✓ Les images sont optimisées (WebP, srcset)",
  "✓ Accessibilité : attributs alt sur images, navigation clavier OK",
  "✓ Variables d'environnement de prod configurées",
  "✓ Build testé en local avant déploiement"
];
```

Anti-patterns React à éviter absolument

ReactJS Avancé — Module approfondi

CONCEPTS CLÉS

La mutation directe du state compromet l'immuabilité, empêchant React d'optimiser la réconciliation via la comparaison des références et générant des bugs difficiles à tracer. L'appel de `setState` dans la phase de rendu provoque des boucles infinies en déclenchant des mises à jour synchrones, perturbant le cycle de vie et la stabilité du virtual DOM. Enfin, le prop drilling massif engendre un couplage fort entre composants, complexifie la maintenance et justifie l'usage de context ou de gestionnaires d'état globaux pour isoler les dépendances et optimiser la granularité des mises à jour.

IMPLÉMENTATION

Exemple de code

```
// Anti-patterns à éviter absolument
// 1. Muter l'état directement
state.items.push(newItem); // ❌
setItems([...items, newItem]); // ✅
// 2. Utiliser l'index comme clé dans une liste dynamique
items.map((item, index) => <li key={index}>{item.name}</li>) // ❌
items.map(item => <li key={item.id}>{item.name}</li>) // ✅
// 3. Fonctions inline dans le render (problème de perf si passé
en prop)
<Child onClick={() => doSomething()} /> // ❌ (Recrée la ref à
chaque render)
const handleClick = useCallback(() => doSomething(), []);
<Child onClick={handleClick} /> // ✅
```

Bonnes pratiques de nommage et conventions

ReactJS Avancé — Module approfondi

CONCEPTS CLÉS

L'adoption rigoureuse de conventions de nommage telles que PascalCase pour les composants React permet d'identifier immédiatement les entités à instancier comme des classes ou fonctions capitalisées, facilitant ainsi la réconciliation du DOM virtuel par React. L'usage de camelCase pour les hooks, notamment préfixés par use, exploite la nature des closures et garantit leur intégration correcte dans le cycle de vie des hooks via l'ordonnement statique, minimisant les risques de rupture du state interne. Par ailleurs, la différenciation stricte des conventions pour les fichiers CSS (kebab-case) optimise la lisibilité en système de fichiers tout en limitant les collisions de noms dans les scopes CSS modulaires, et l'application cohérente de noms explicites pour les actions Redux et les types TypeScript renforce la traçabilité des flux d'état et la robustesse typée dans les chaînes de traitements asynchrones.

IMPLÉMENTATION

Exemple de code

```
// Conventions de nommage React
// Composants : PascalCase
const UserProfile = () => <div />;

// Fichiers : PascalCase pour les composants, camelCase pour
// utilitaires
// UserProfile.tsx, useAuth.ts, mathUtils.ts
// Props : camelCase, préfixées par 'on' pour les événements
<Button onClick={handleClick} isLoading={true} />

// Hooks : préfixés par 'use'
const useWindowSize = () => { /* ... */ };

// Types/Interfaces : PascalCase
interface UserProps { id: string; }
```

Récapitulatif des commandes essentielles

Commandes npm et outils pour un workflow React + TypeScript optimal

Tableau des commandes npm, descriptions et contextes d'utilisation

Commande	Description	Contexte
npm create vite@latest my-app -- --template react-ts	Créer un projet React+Vite+TypeScript	Démarrage
npm install	Installer les dépendances	Setup
npm run dev	Lancer le serveur de développement	Développement
npm run build	Compiler pour la production	Build
npm run preview	Prévisualiser le build	Validation
npm run lint	Vérifier le code avec ESLint	Qualité
npx tsc --noEmit	Vérifier les types TypeScript	Qualité
npm test	Lancer les tests Jest	Tests
npm test -- --coverage	Tests avec rapport de couverture	Tests
npm test -- --watch	Tests en mode watch	Développement
npx playwright test	Tests E2E	Tests

Synthèse — Ce qu'il faut retenir

Récapitulatif des compétences acquises durant la formation

Module	Thème principal	Concepts clés maîtrisés
Module 1	Introduction & Écosystème	Rappels ES6+, VirtualDOM, JSX, One-way Data Flow, Vite.
Module 2	Bonnes pratiques	Typage strict avec TypeScript, Tests unitaires avec Jest et React Testing Library.
Module 3	Techniques & Patterns Avancés	Higher Order Components (HOC), Context API, Portals, Custom Hooks, useEffect avancé.
Module 4	Redux Avancé	Redux Toolkit (createSlice), Immer, Redux Persist, Middlewares personnalisés.
Module 5	Optimisation des Performances	Code Splitting (React.lazy, Suspense), Mémoïsation (useMemo, useCallback, React.memo).
Module 6	Animations & Transitions	React Transition Group, CSSTransition, Transitions de routes fluides.
Module 7	Internationalisation (i18n)	react-i18next, useTranslation, Changement de langue dynamique, Fichiers de ressources.

Conclusion & Prochaines étapes

Bilan de la formation et perspectives d'évolution

Compétences acquises

Architecture avancée

Maîtrise des patterns HOC, Portals, Context API et création de Custom Hooks robustes.

Optimisation des performances

Réduction du bundle avec React.lazy et prévention des re-renders inutiles (useMemo, useCallback).

Qualité et fiabilité du code

Typage strict avec TypeScript et couverture de tests automatisés avec Jest et React Testing Library.

Écosystème moderne

Intégration de Redux Toolkit, gestion des animations fluides et internationalisation (i18n).

Prochaines étapes recommandées

Data Fetching avancé

Explorer **React Query** ou **SWR** pour la gestion du cache, la synchronisation et les requêtes complexes.

Server-Side Rendering (SSR)

Se former à **Next.js 14+** (App Router) pour le rendu côté serveur, le SEO et les Server Components.

Gestion d'état alternative

Découvrir des solutions plus légères que Redux comme **Zustand** ou **Jotai** pour les petits projets.

Pratique et Open Source

Contribuer à des projets open source React ou mettre en place une architecture Micro-Frontends.

Merci de votre attention

Avez-vous des questions ?

CONTACT ORSYS

inscription@orsys.fr • 01 49 07 73 73